

APAI LAB 04: PULP-NN - Deep Neural Network Inference on PULP

Călin Diaconu – calin.diaconu2@unibo.it
Ahmad Mirsalari – seyedahmad.mirsalar2@unibo.it
Ihor Sachok – ihor.sachok@unibo.it



01. Lab Information

01. Lab Information – Objective of the Class

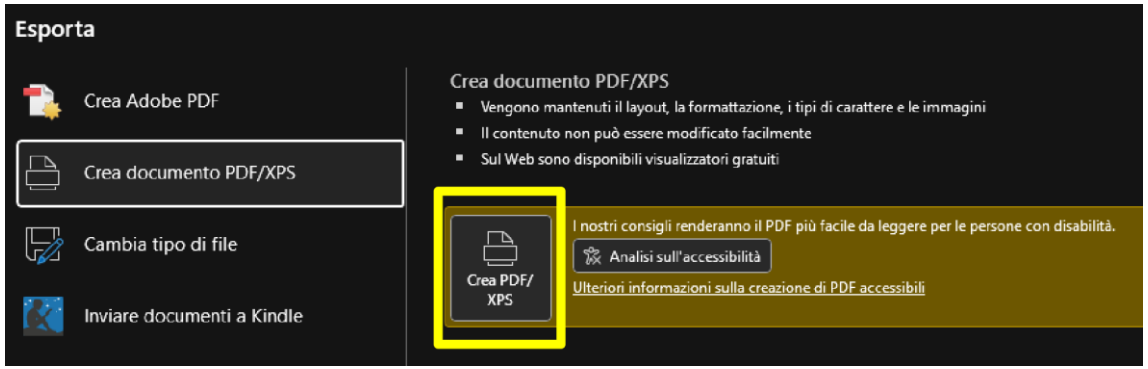
- **Subjects**:
  - Parallelization on the PULP architecture
  - Matrix multiplication
  - Fully connected layer
  - 2D convolution
  - Profiling code execution
- **Programming language**: C
- **Lab duration**: 3h

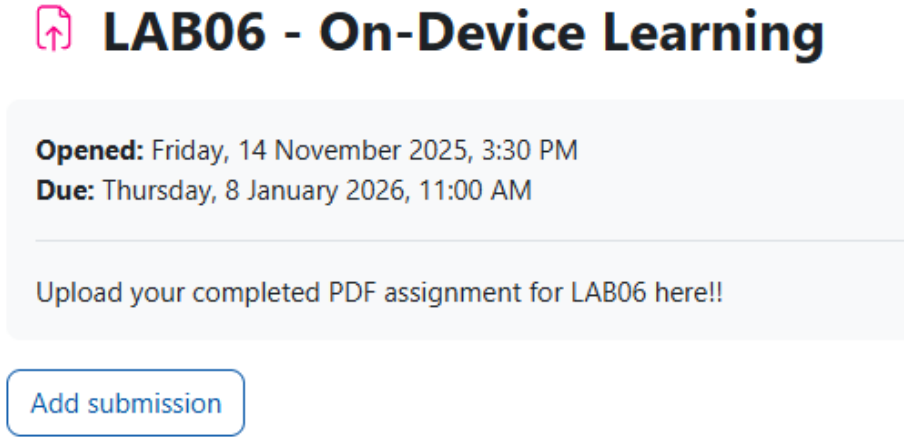
- **AI policy**: we will not prevent you from using AI in solving the assignments, but the lab is meant to be a hands-on learning experience. To keep it useful, we have included an .md file (also used in a Stanford class), that should be automatically read by AI agents, in order to prevent them to give you a direct solution and rather guide you towards finding it yourself.
- The class is meant to be interactive: coding together and on your own!

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

01. Lab Information – How to Deliver the Assignment

- You will deliver **ONLY THE PDF** assignment, **NO code**
  - Download the **deliverable template** from Virtuale
  - **Fill in the results** required by the assignment
  - **Export to PDF** format (this option is available in most document editors – Google Docs, Office Word, or as a “Print to PDF” option in the Print menu)
  - **Rename the file** to *LAB<number_of_the_lesson>_APAI_<name_surname>.pdf*
  - Use the **Virtuale platform to upload the PDF** before the **deadline (Sunday 8th of November 2026, end of day, 23:59)**
  - For each lab that you solve and submit successfully, you will receive **1 extra point**, up to a **maximum of 4 points**





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



02. Prerequisites

02. Prerequisites – Containers & Docker



STEPS ONLY FOR PERSONAL MACHINES

- **What is a Container?**
  - A lightweight, portable, and self-contained environment that bundles together
    - A consistent runtime environment
    - Your application code
    - All its dependencies (libraries, tools, configurations)
- **What is Docker?**
  - The most used platform for working with containers, which allows the user to:
    - **Build images**, defined in a Dockerfile
    - **Run containers** – isolated environments, created according to the "recipe" defined inside an image
    - **Share images** to other people through the Docker Hub, for reuse and consistency in development
- The **Docker image** used in this course includes the PULP-SDK and its related compilation toolchain

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Docker Setup (WSL install)



STEPS ONLY FOR PERSONAL MACHINES

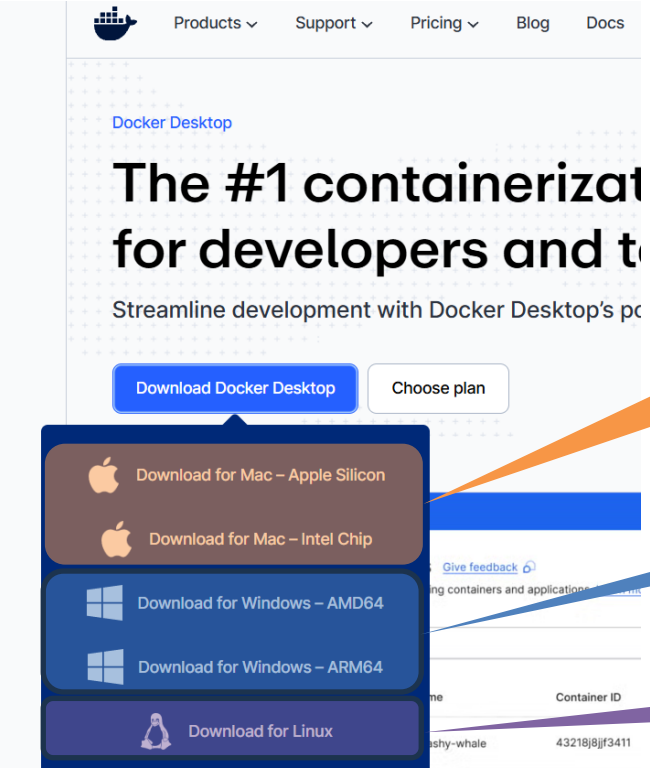
- **Step 0** (for **Windows users only!**) – install Windows Subsystem for Linux (WSL 2)
  - **What is WSL?**
    - **A compatibility layer** that lets you run a genuine Linux environment directly on Windows, without a separate virtual machine or dual boot
    - Gives access to Linux command-line tools and applications, running natively alongside your Windows app
    - It can run both command-line and graphical Linux apps directly inside Windows
  - The **advantage** of running Docker through WSL is that, while in both cases Docker runs through a virtual machine, the WSL is much lighter and it doesn't slow-down the entire system
  - If you plan to work on the lab on a **Windows** machine, please **follow these installation instructions for WSL**

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Docker Setup

STEPS ONLY FOR PERSONAL MACHINES

- **Step 1 – install Docker Desktop**
  - Choose, download and install the proper version from the official website.



- Apple Mac versions
- Choose based on your computer's processor

- Windows versions (AMD64 for most, unless you are **VERY SURE** that your processor is ARM64)
- **CHOOSE THIS EVEN IF YOU WORK WITH WSL!**

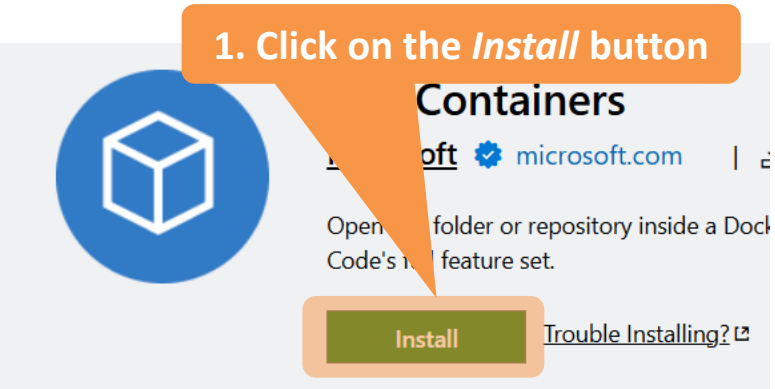
- Universal Linux version
- **If working with WSL, choose a Windows version!**

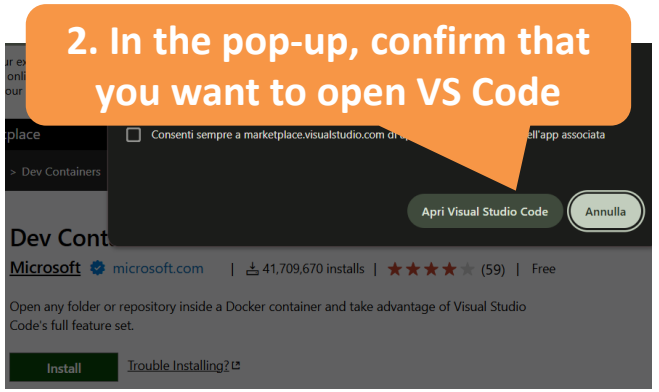
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

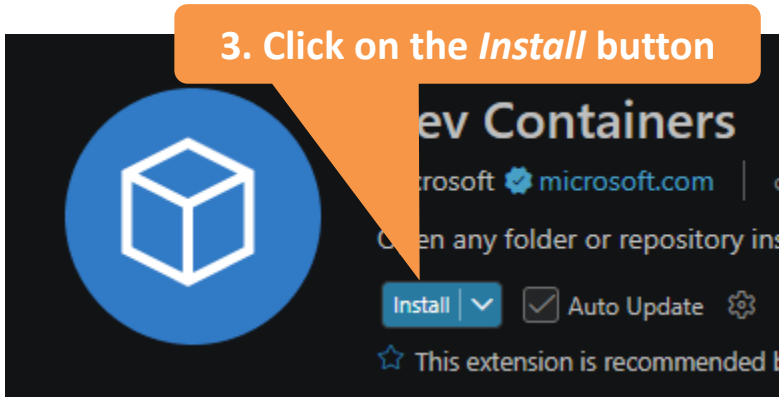
02. Prerequisites – VS Code

STEPS ONLY FOR PERSONAL MACHINES

- **Step 2 – Install Virtual Studio (VS) Code**
  - **What is VS Code?**
    - A free, popular and versatile text editor from Microsoft
    - It is highly customizable through a large extension library
    - The one that interests us is the **Dev Containers** extension, that enables VS Code to work with Containers
  - **How to install?**
    - Choose, download and install for your operating system from the official website
    - **Once VS Code installed**, we need to install the extension for working with Containers
      - Find and install it from the VS Code Marketplace







ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Error Solving

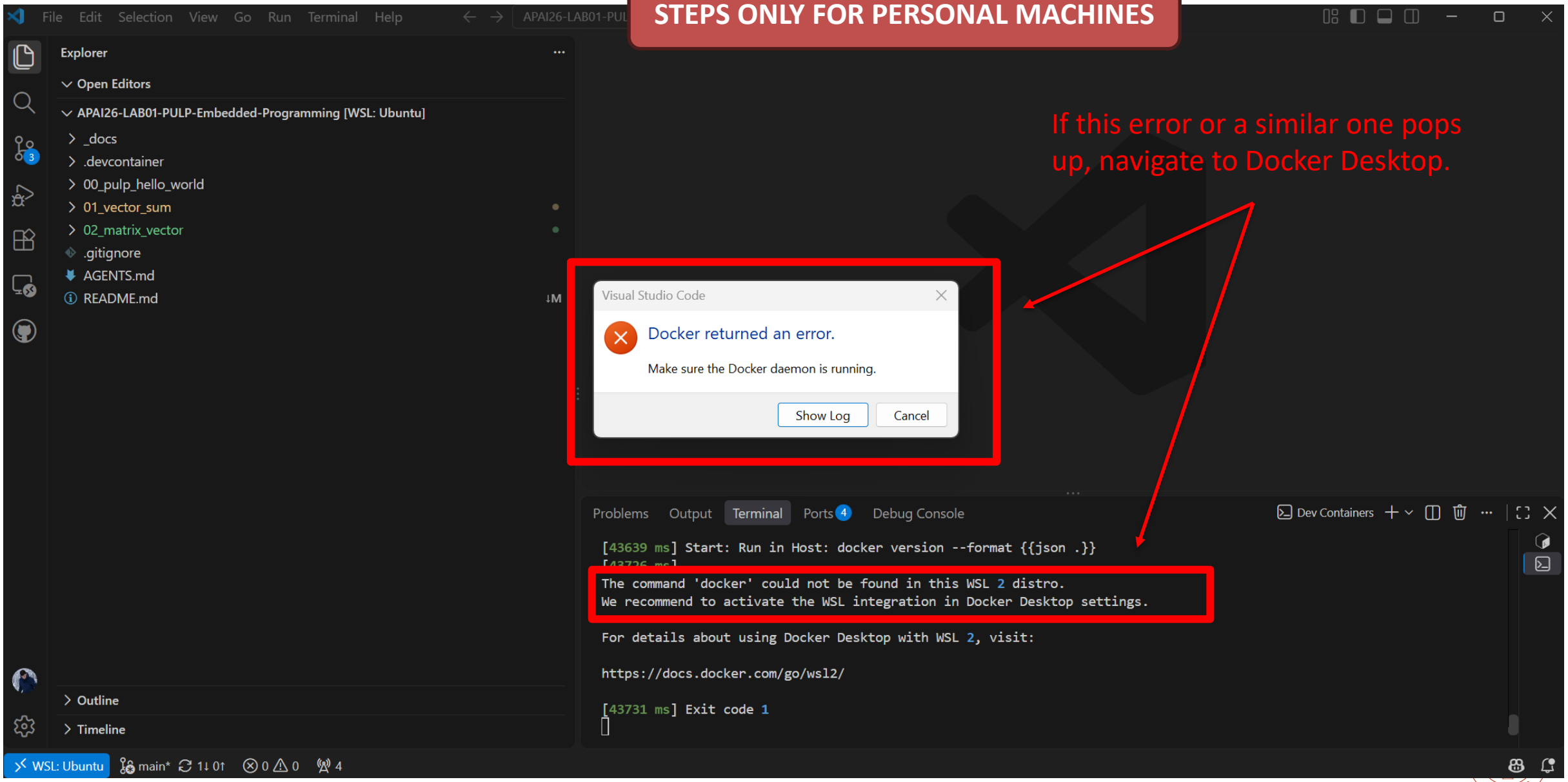
STEPS ONLY FOR PERSONAL MACHINES

IN VS CODE, IF DOCKER RETURNS AN ERROR,

FOLLOW THE INSTRUCTIONS IN THE FOLLOWING SLIDES

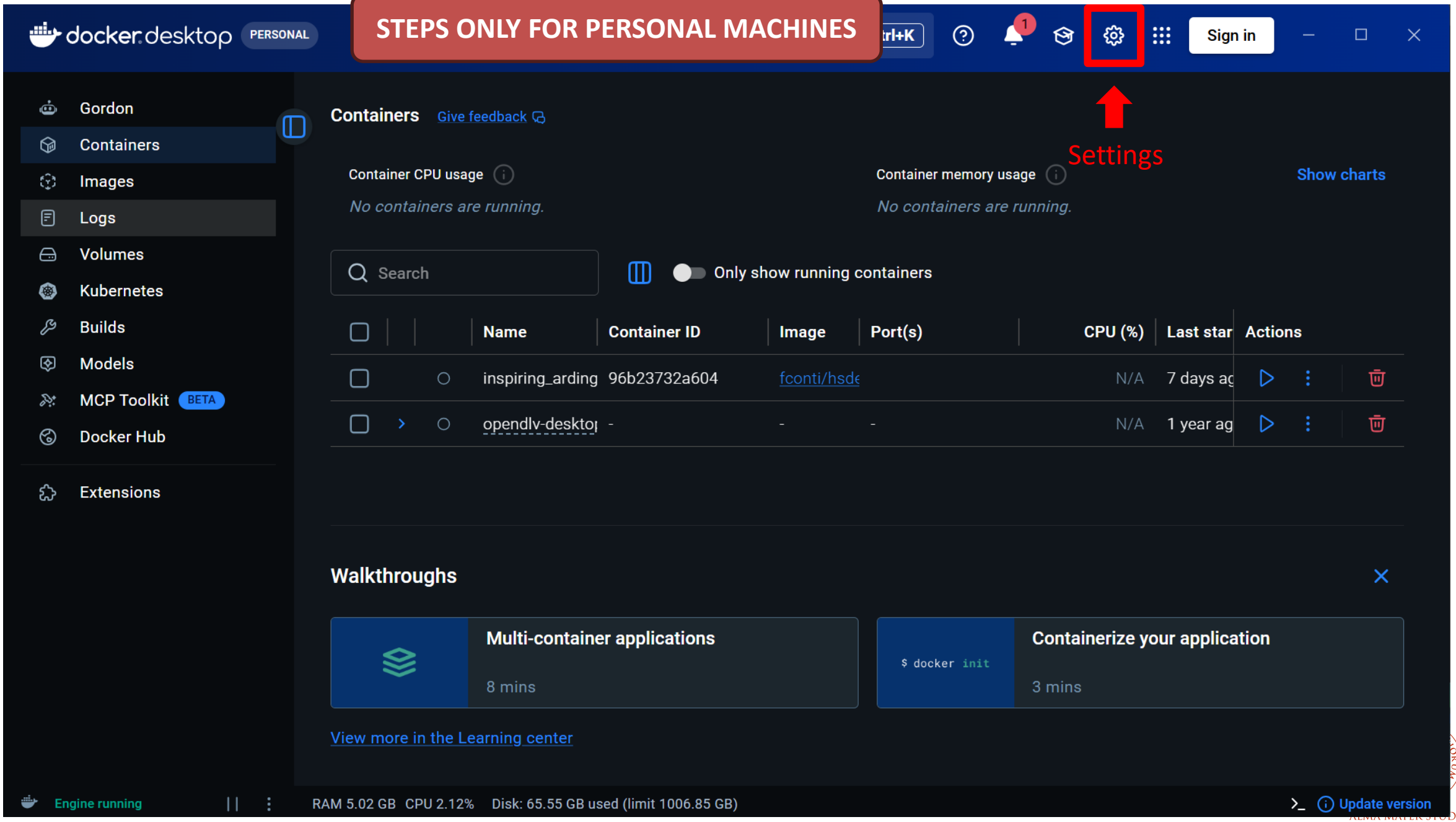
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

STEPS ONLY FOR PERSONAL MACHINES

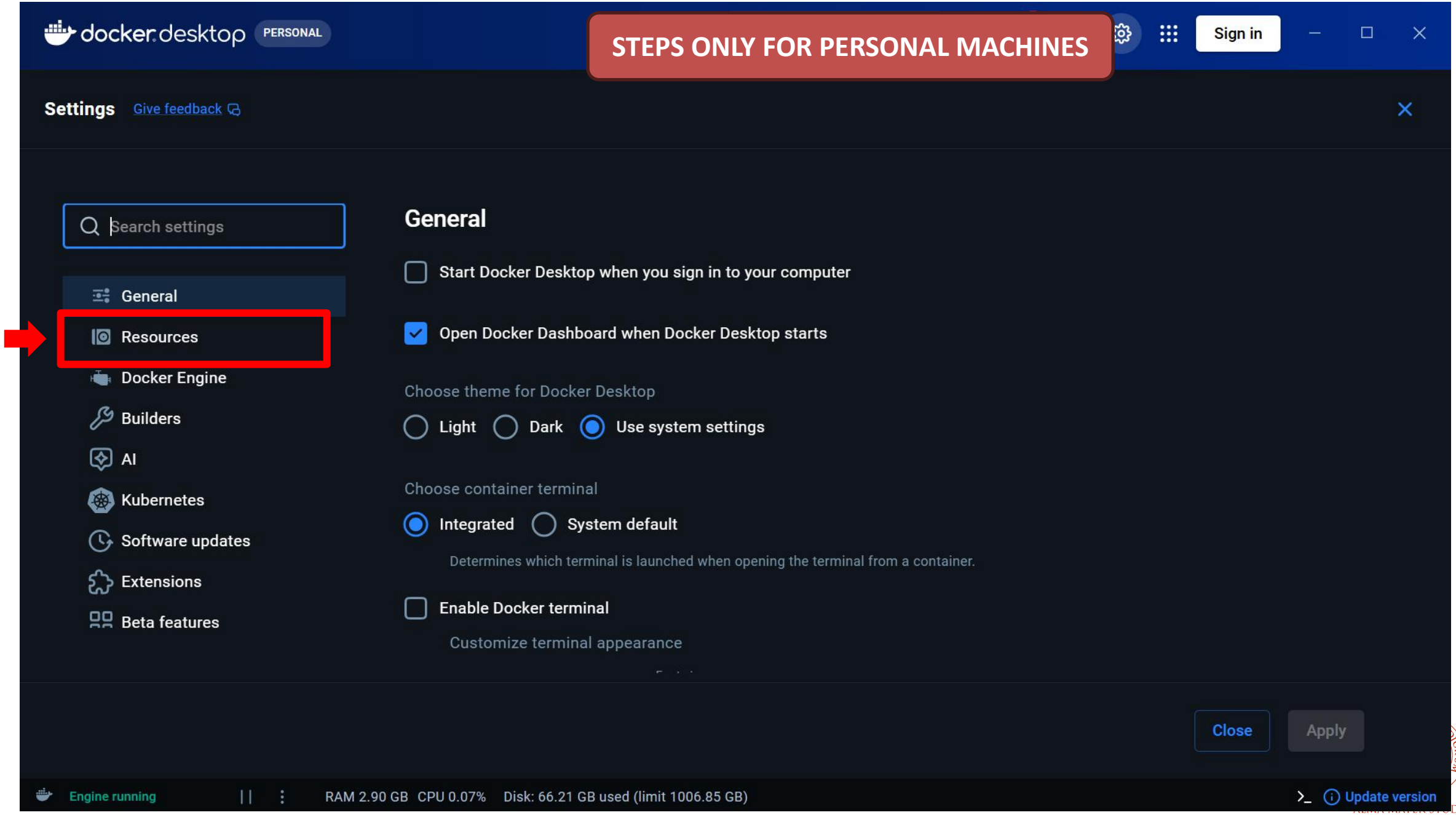


If this error or a similar one pops up, navigate to Docker Desktop.

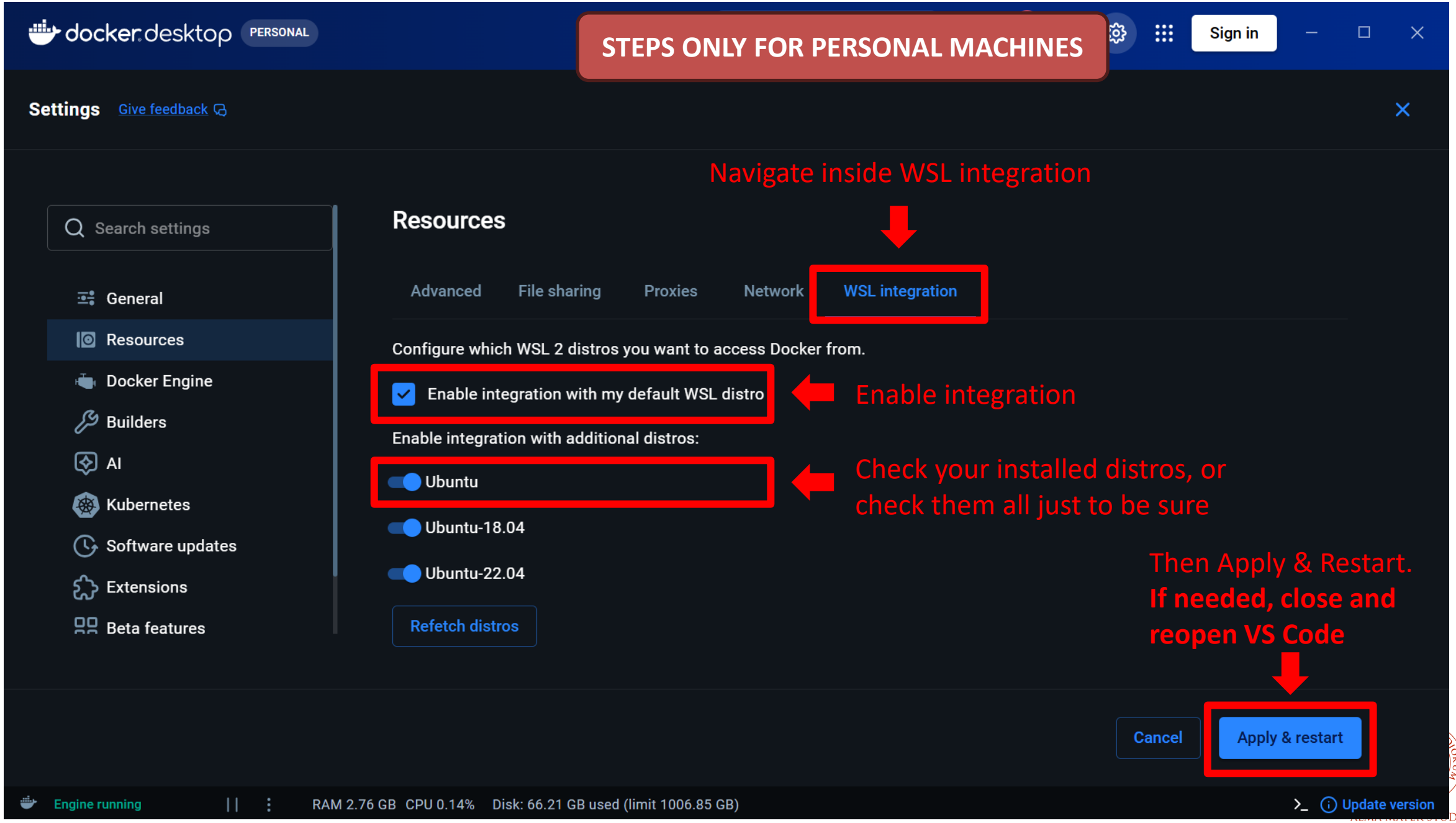
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

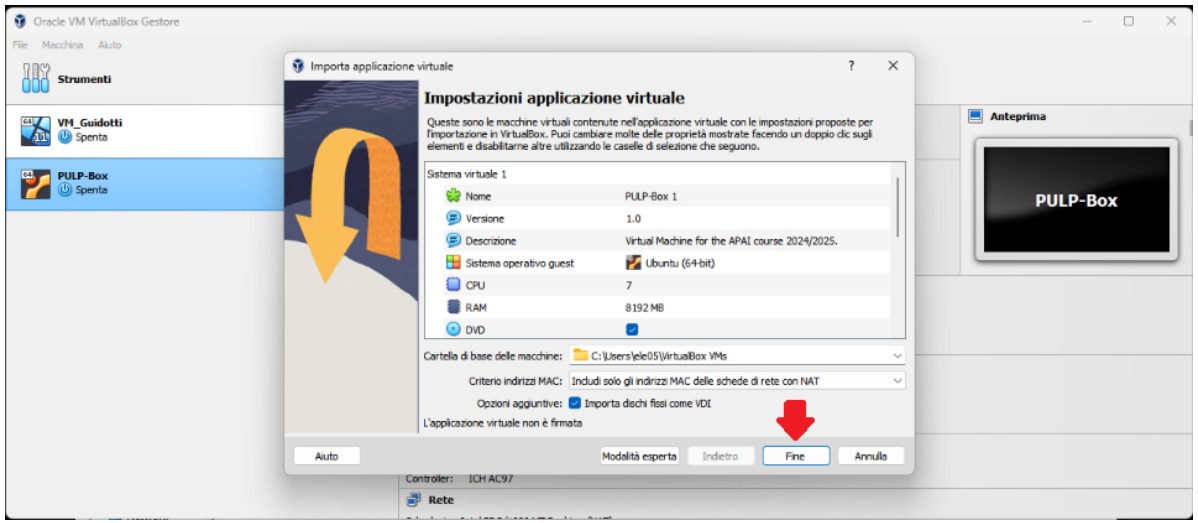
02. Prerequisites – Virtual Machine – Part 1

STEPS ONLY FOR LAB MACHINES

- When using the **lab computers**, you will find an Ubuntu-based virtual machine that already contains everything that is needed for solving the assignments:
  - **PULP-SDK –** so we don't need the Docker container anymore
  - **VS-Code –** in this case, only used for code editing and accessing the terminal
- You will only be required to **download the resources** for the lab, which are also **on Virtuale** (the only website accessible from the lab computers) as a **zip file**. This must be done from a browser **OUTSIDE of the virtual machine**. This is also a good step to **unzip** it, with a simple *right-click -> Extract*

Once you have this zip file, follow these steps:

- **Step 1**: Go to *C:/VM_APAI* and double-click *PULP-box.ova*. When VirtualBox opens, click on *"Finish"/"Fine"*

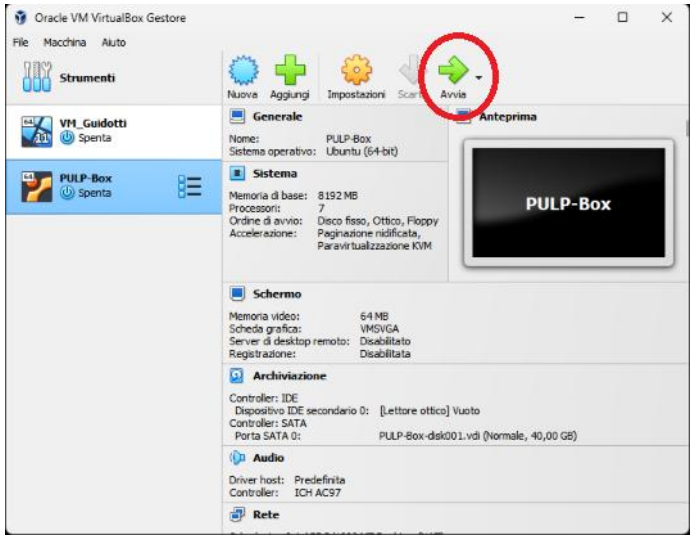


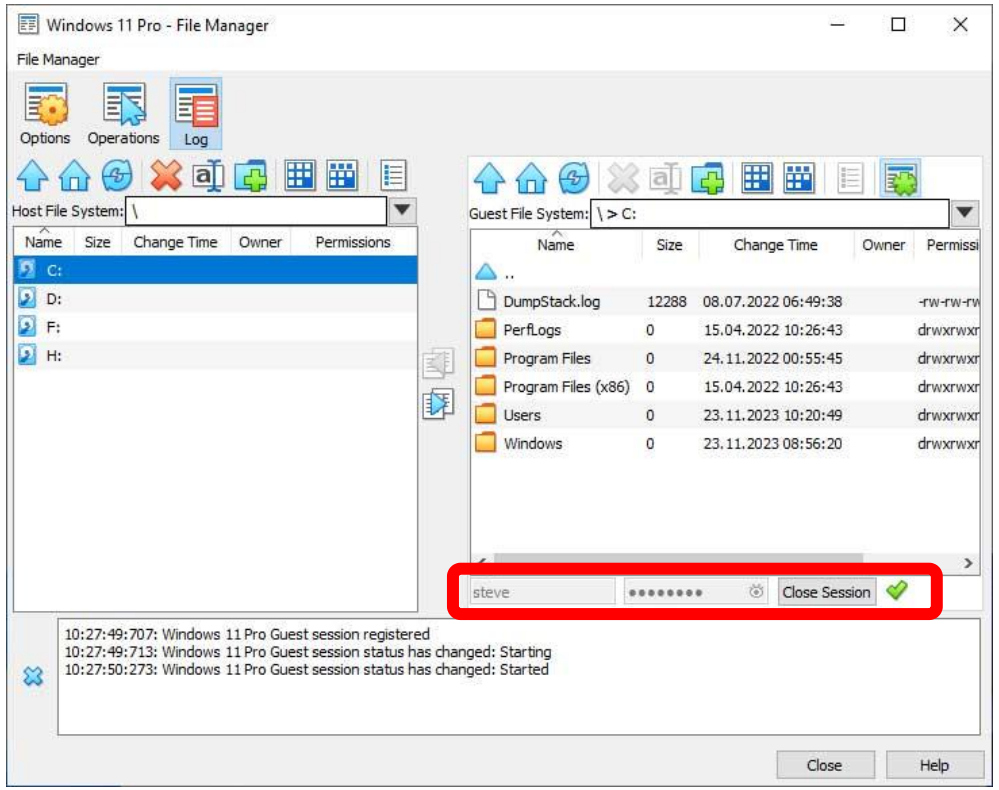
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Virtual Machine – Part 2

STEPS ONLY FOR LAB MACHINES

- **Step 2**: Once the VM is imported, open it by clicking on *"Start"/"Avvia"*
- **Step 3**: For login – the username is ***pulp-box*** and password is ***pulp***
- **Step 4**: Access the files of the VM:
  - From the **top-bar menu**, access ***Machine -> File Manager***
  - In the **left** panel, access files from the **PC**, in the **right**, access files from the **VM**
  - In the **bottom of the right panel**, you will have to **enter the login credentials**





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Virtual Machine – Part 3

STEPS ONLY FOR LAB MACHINES

- **Step 5**: With a simple drag-and-drop, **move the files from the PC location into a VM** location you choose (it must be a subdirectory of the home directory to have file writing rights)
- **Step 6**: Inside the VM, open a terminal and set up the PULP SDK by running

```
source setup_pulp_sdk.sh
```

- **Step 7**: Go to the VM location you chose to put the assignment data in, using the *cd* command in the terminal

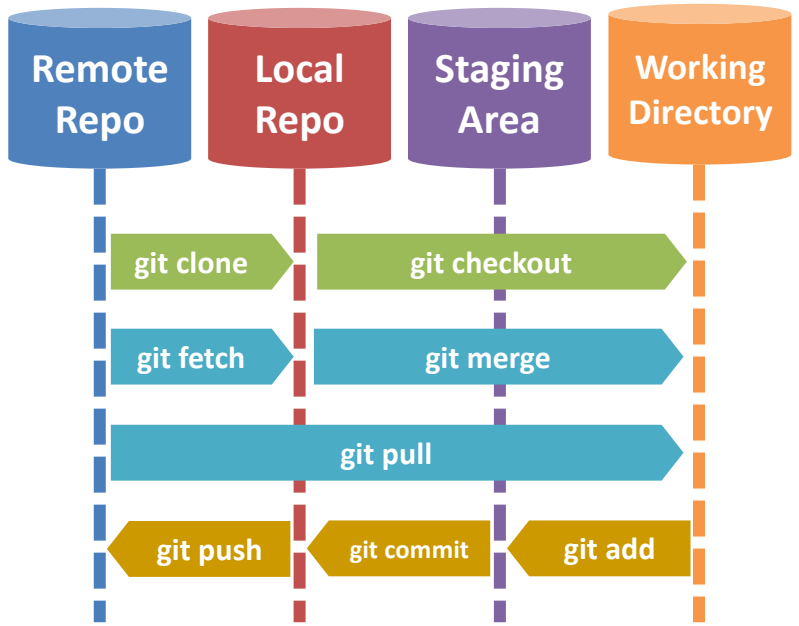
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – git

Link to a useful intro git tutorial



- **What is Git?**
  - A free, open-source version control system that tracks changes to files over time.
  - It lets multiple people work on the same codebase (repository) and merge changes (commits) without overwriting each other's work.
  - It keeps a full history of changes, so you can review, compare, or revert to any earlier version.
- **Common commands we will use**
  - **git clone –** Downloads a full copy of a remote repository to your machine
  - **git checkout** – Switches your working directory to a different branch or commit
  - **git fetch –** Checks the remote for new changes without applying them locally
  - **git merge –** Combines changes from one branch into another
  - **git pull –** Fetches (*downloads)* remote changes and merges them into your local branch (equivalent to *git fetch* + *git merge*)
  - **git add [file1, file2, …] –** Stages (*marks*) the specified files to be included in the next commit
  - **git commit –m "commit message" –** Saves the staged changes to your local repository's history with a message
  - **git push –** Uploads your committed changes to the remote repository



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

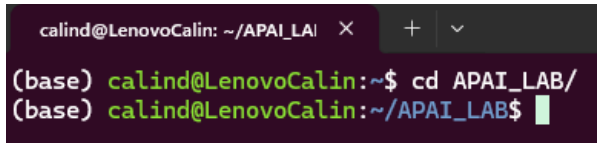
02. Prerequisites – Hello, World Example – Part 1

STEPS ONLY FOR PERSONAL MACHINES

In order to confirm that everything was installed correctly, we will now run the ***Hello, World*** basic example available in the repository of this lab.

The **example is run on WSL, on a Windows** machine, but it should work identically on other operating systems.

- **Step 1:** In a terminal, open the location where you wish to work



- **Step 2:** Clone the repository of lab 1 locally

```
git clone https://github.com/EEESlab/APAI26-LAB04-PULP-NN
```

- **Step 3:** Enter the cloned directory

```
cd APAI26-LAB04-PULP-NN/
```

- **Step 4:** Open the directory in VS Code

```
code .
```

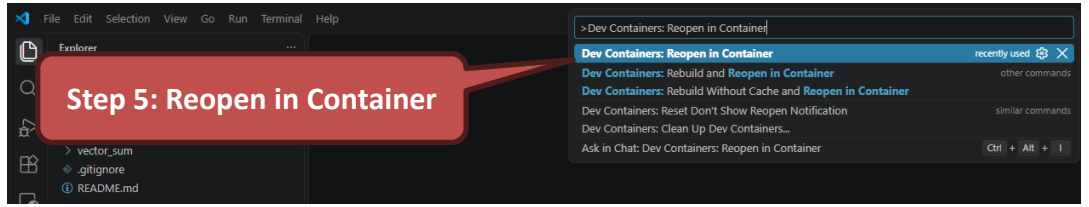

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Hello, World Example – Part 2

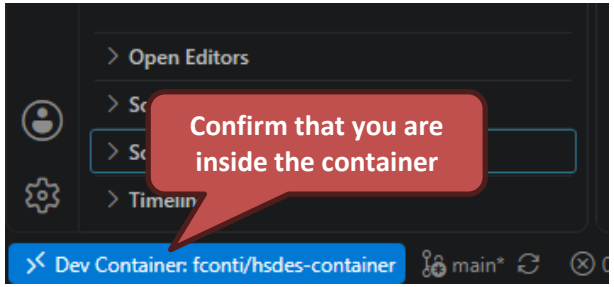
STEPS ONLY FOR PERSONAL MACHINES

* From this point on, we will only work from VS Code, so it is ok to close the terminal you have previously opened.

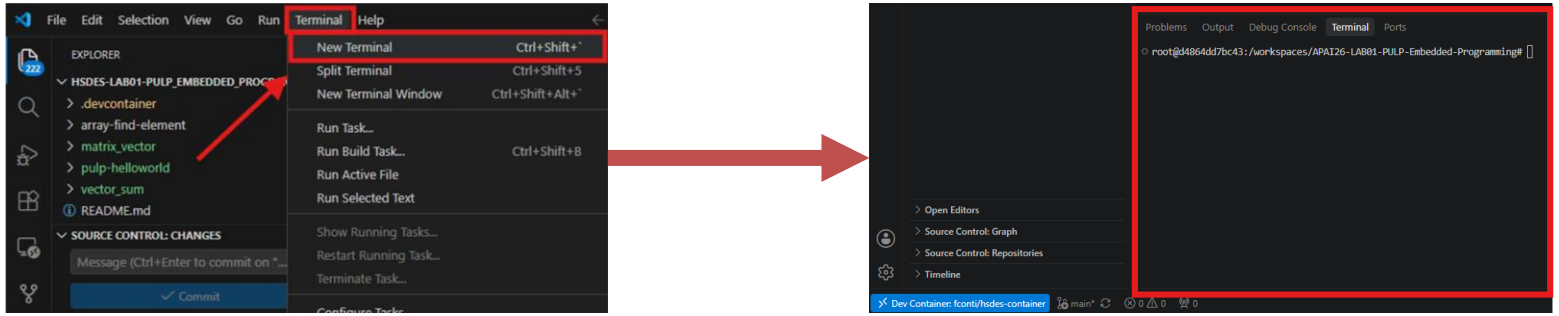
- **Step 5:** At this point, the project is open in the local *environment*. We need to **reopen inside the Docker container**.
  - **Step 5.1**: Press **CTRL** + **Shift** + **P**
  - **Step 5.2**: Type in ***"Dev Containers: Reopen in Container"***
  - **Step 5.3**: Press **Enter**



* We are now **inside the container**. From now on, when opening VS Code, the project will **automatically open inside the Docker Container**. It is confirmed in the bottom-left corner of VS Code.



- **Step 6:** Open a **terminal inside VS Code** (which will be **inside the container**)
  - Access the terminal through the **top-left task bar (*Terminal -> New terminal*)**
  - The terminal **will open in the bottom** of the window.



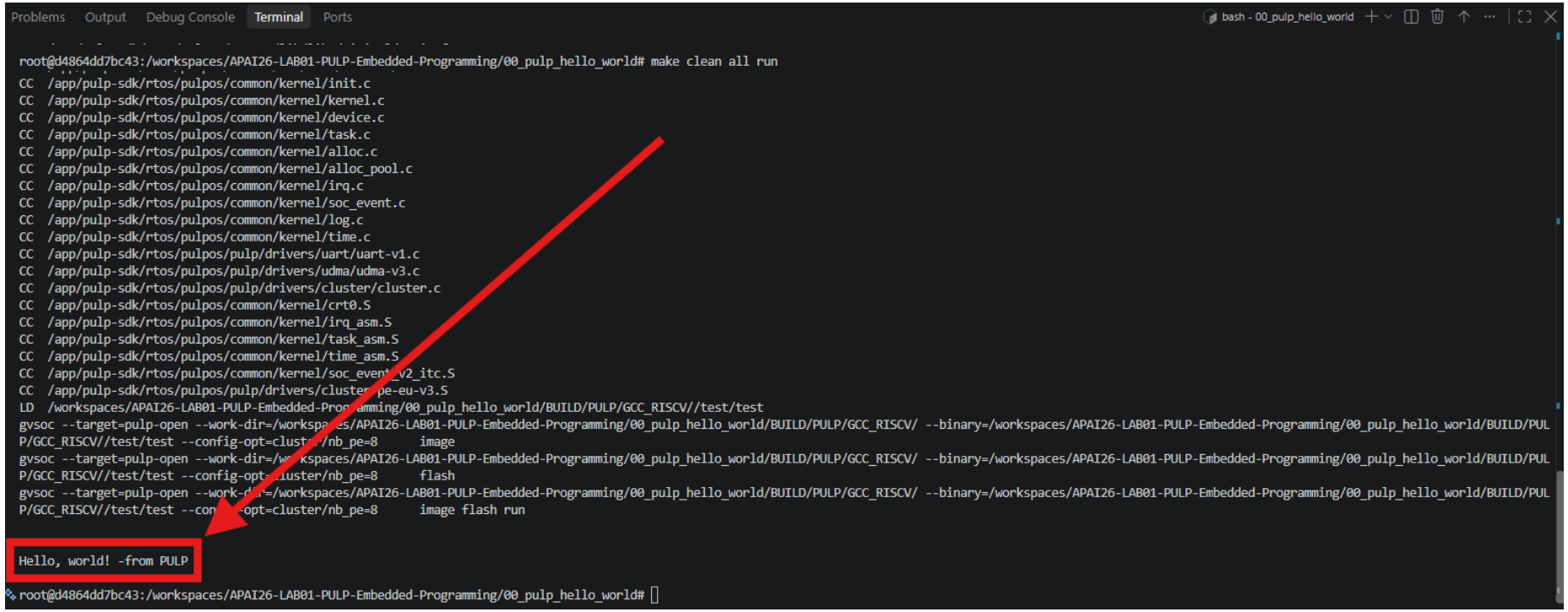
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Hello, World Example – Part 3

- **Step 7:** Run the ***Hello, World*** example

```
cd 00_pulp_hello_world/
make clean all run
```

- If everything went well, you should now see the "*Hello, world! -from PULP*" message in the VS Code terminal.



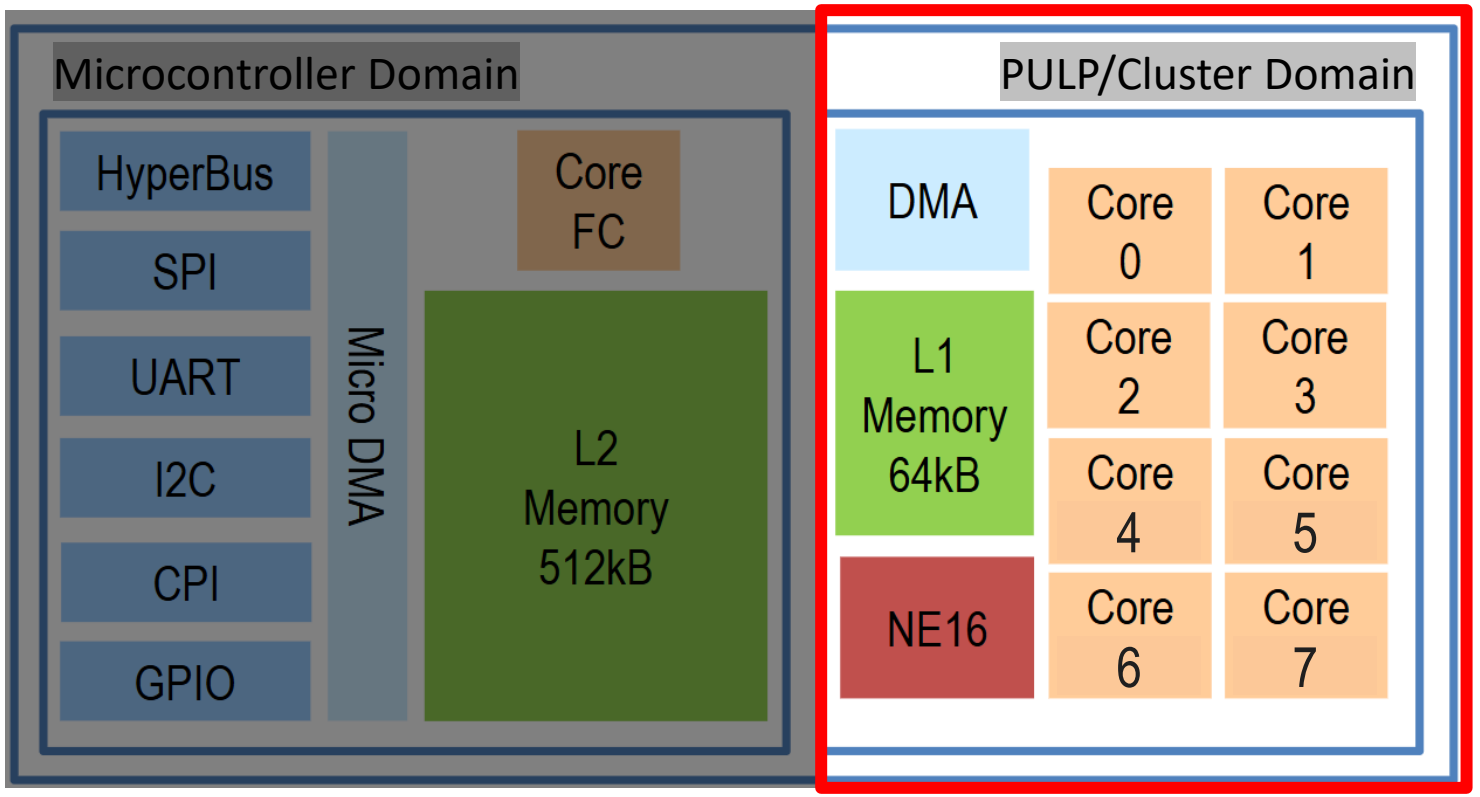
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



03. Introductory Information

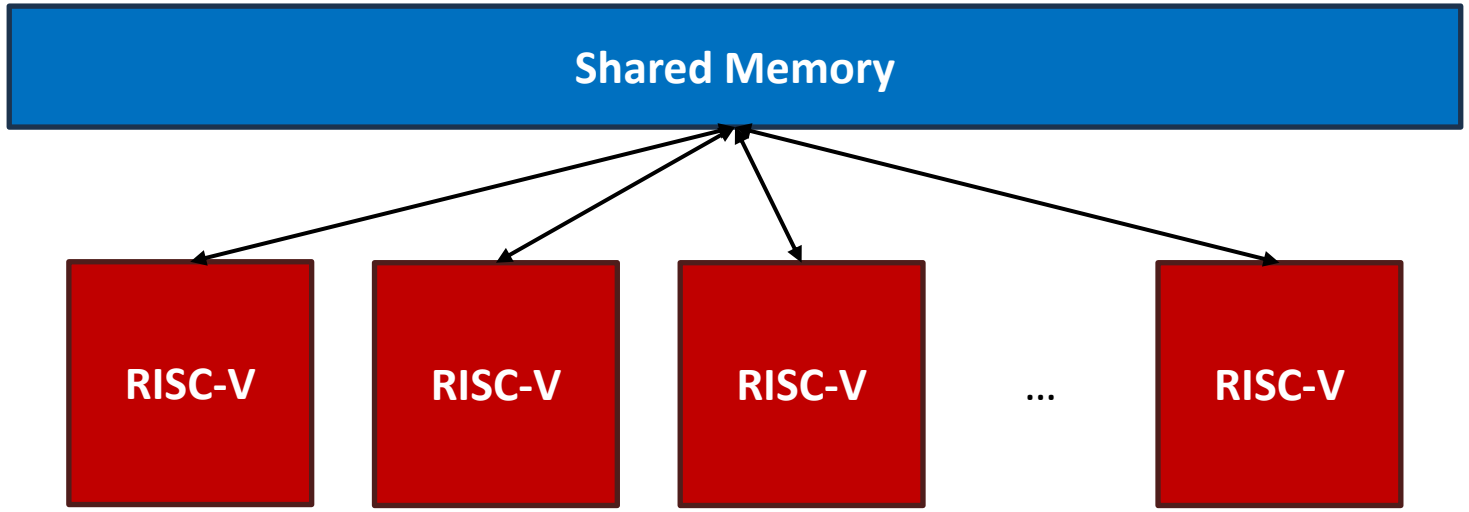
03. Intro Information – Hardware Context: The PULP Platform

- Today we focus on the 8-core cluster
  - **Cores**: 8 high-performance, with extended RISC-V ISA
  - **On-chip Memory: level 1** memory shared only by the cores in Cluster
  - **cluster-DMA**: multi-channel 1D/2D cluster-DMA that controls the transactions between L1 and L2 Memory



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

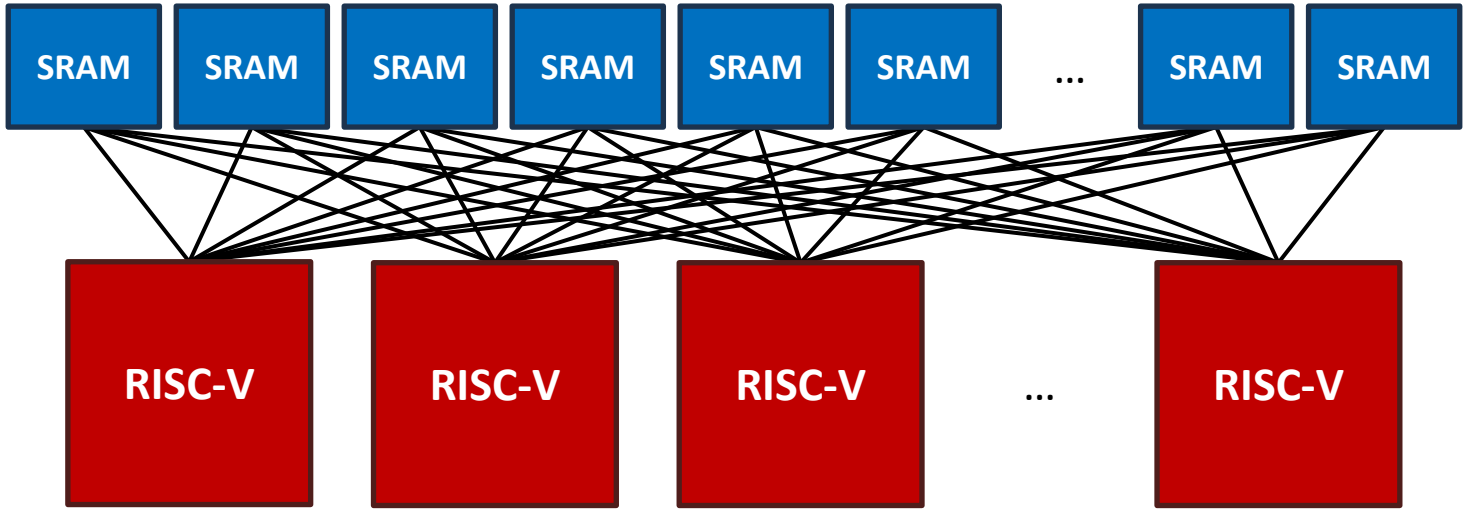
03. Intro Information – Hardware Context: PULP Cluster



Target: a **Shared-Memory** parallel programming model with 4—16 DSP cores sharing directly an **L1 Memory (Tightly Coupled Data Mem or TCDM)**

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

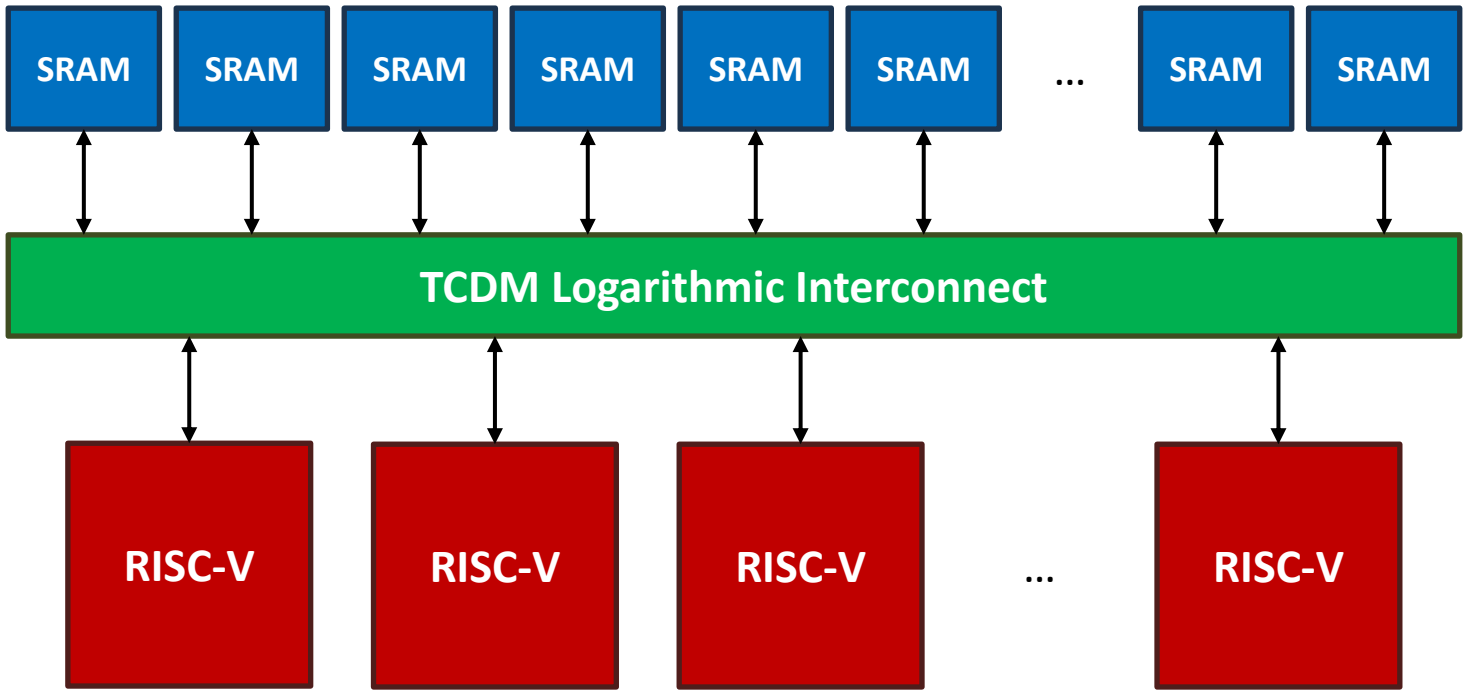
03. Intro Information – Hardware Context: PULP Cluster



Memory is organized in **multiple banks** to enable **concurrent access**

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

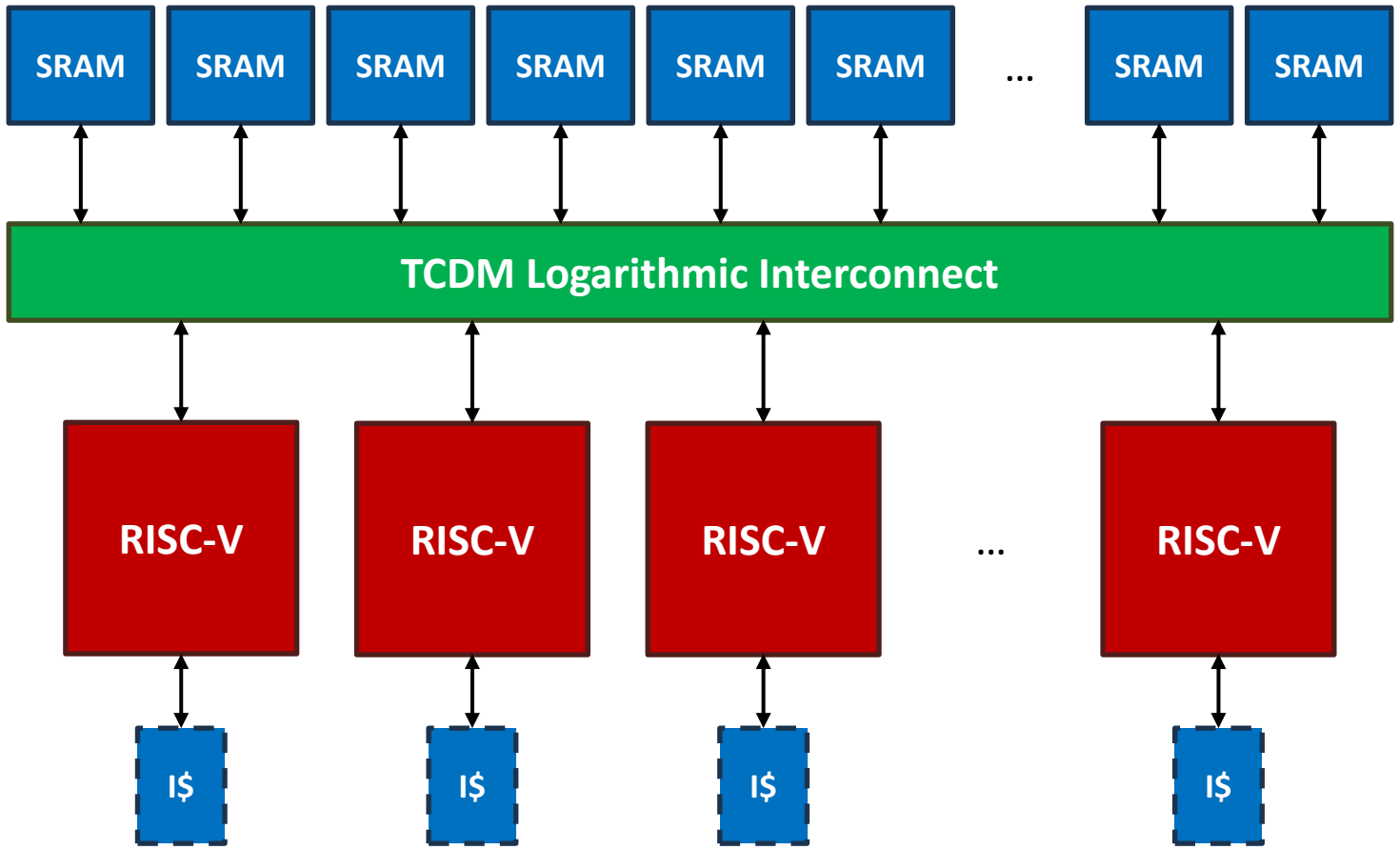
03. Intro Information – Hardware Context: PULP Cluster



Introduce the Interconnect for full connectivity (**1-cycle Access Crossbar**)

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

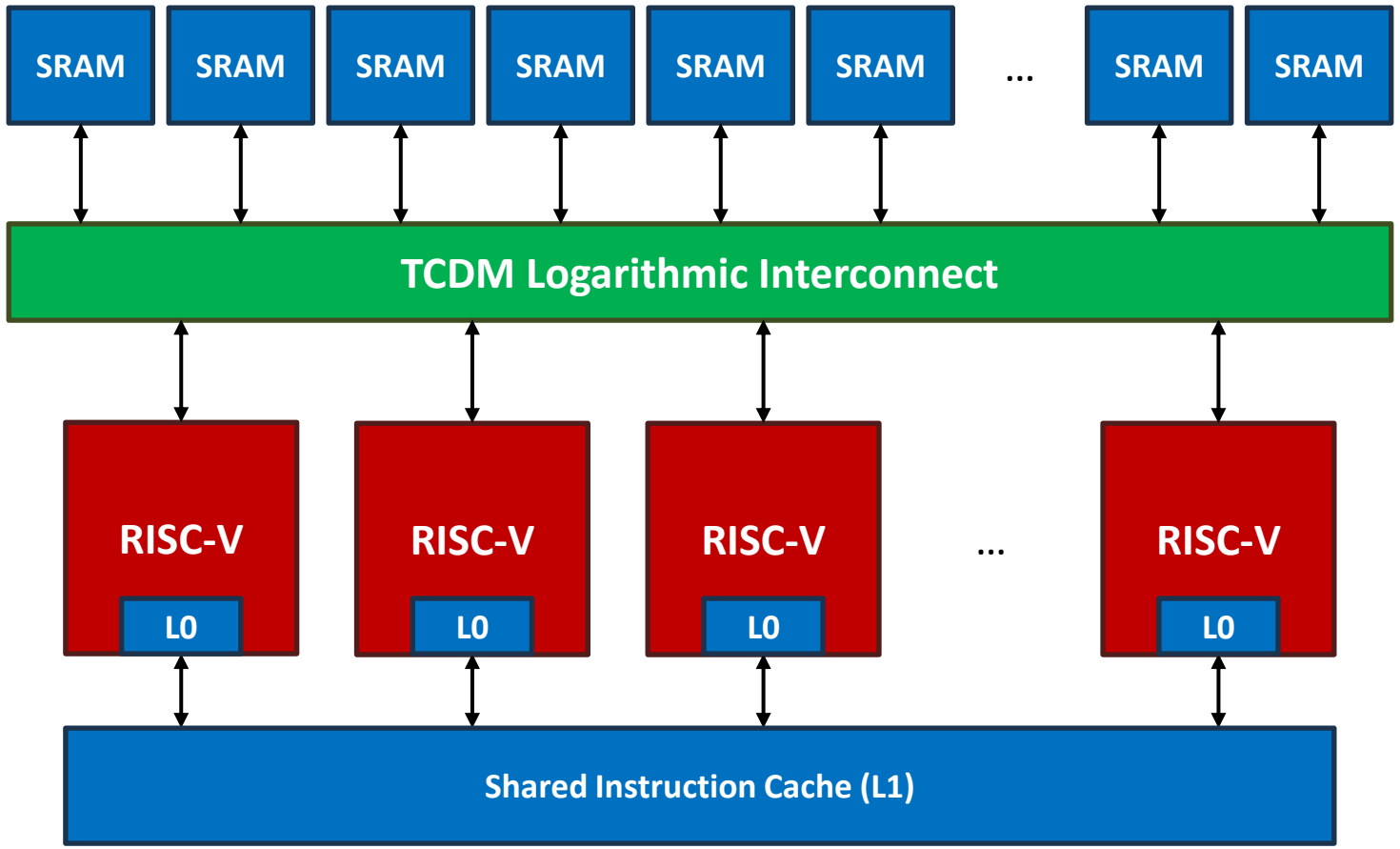
03. Intro Information – Hardware Context: PULP Cluster



Multiple **separate instruction caches** (hardware threads):
MIMD execution scheme

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

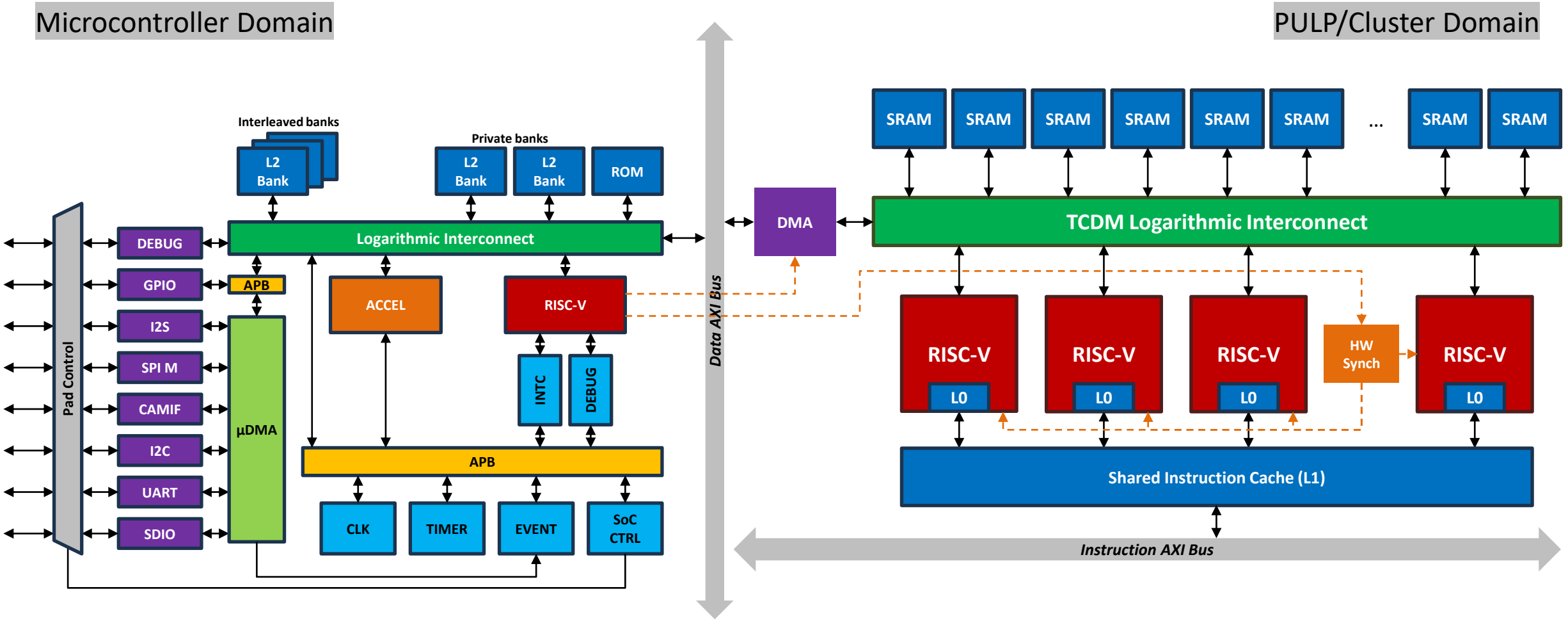
03. Intro Information – Hardware Context: PULP Cluster



But **optimized for a single parallel program** (**SPMD**).
SPMD executes **multiple instances** of the **same program independently**,
where each program works on a **different portion of the data**.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

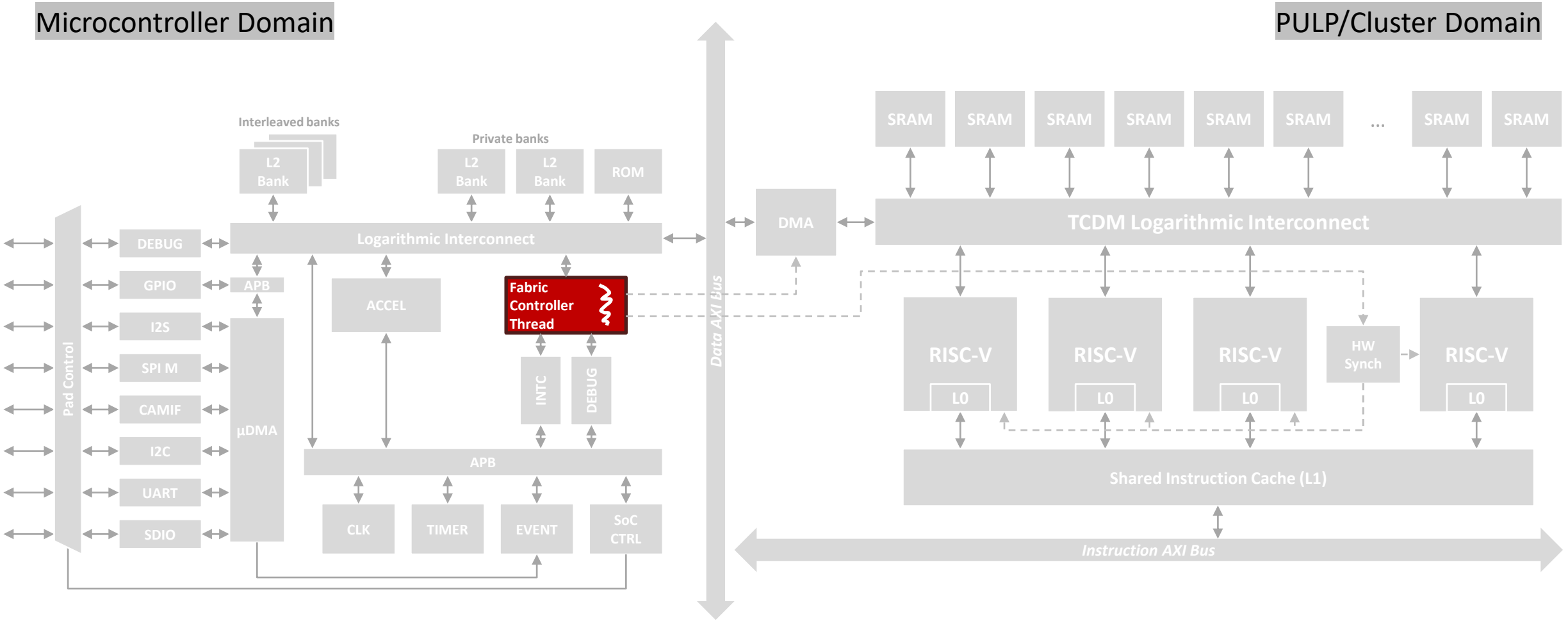
03. Intro Information – Hardware Context: PULP Cluster in the PULP Platform



Data movement is **fully software-managed**

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

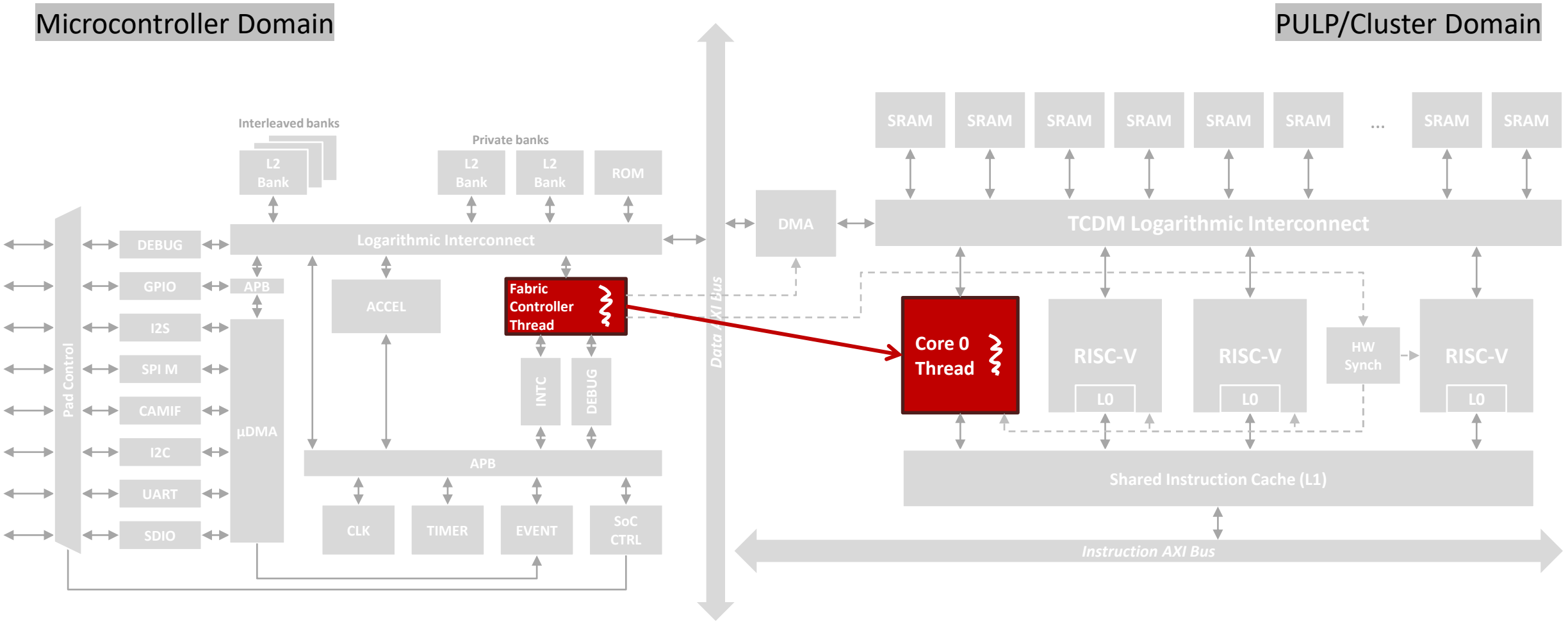
03. Intro Information – PULP Execution Model



Cluster is **inactive** and **clock-gated at boot**:
a **single thread runs on the Fabric Controller (MCU core)**

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

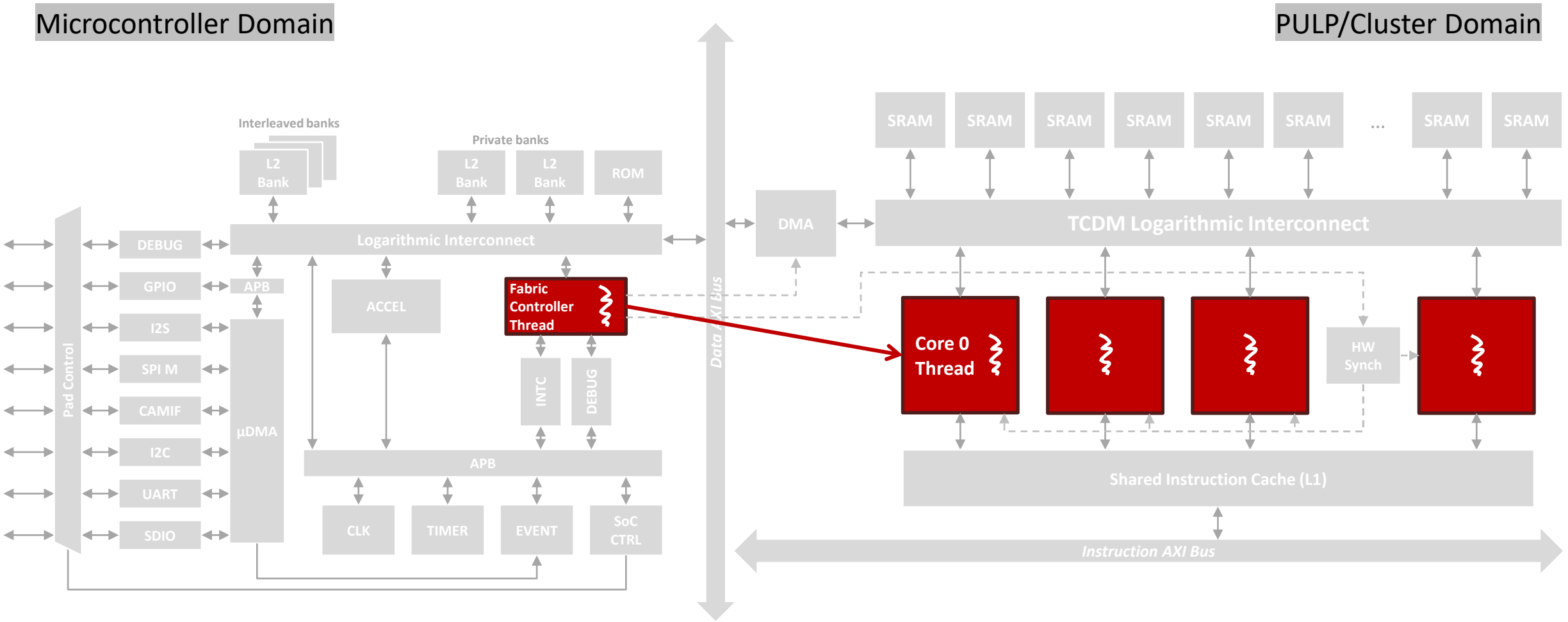
03. Intro Information – PULP Execution Model



Activate the cluster and **call a function** on cluster **core 0**

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – PULP Execution Model



Fork an **execution team** on multiple cluster cores.
They can be synchronized with **barriers, critical sections**.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – PULP Execution Model in Software – PMSIS & Synchronization

- **PMSIS –** PULP Microcontroller Software Interface Standard

- How to **manage a device lifecycle:**
  - Configuration and initialization (*device specific*)
    - **conf_init()**
    - **open_from_conf()**
  - Prepare the device for usage
    - **open()**
  - Perform required operations (*device specific*)
  - Release the resources
    - **close()**

- **Barriers** (*used for intermediate and final join points; all cores wait for the others to reach this point*):
  - **pi_cl_team_barrier();**

- **Critical sections** (*used to avoid critical races; only one core can enter this section at a time*):
  - **pi_cl_team_critical_enter();**
  - *// code in the critical section*
  - **pi_cl_team_critical_exit();**

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – PMSIS – Using the Cluster as a Device

- You will find this content in *main.c*

```
struct pi_device cluster_dev = {0};
struct pi_cluster_conf conf;
struct pi_cluster_task cluster_task = {0};

// task allocation
pi_cluster_task(&cluster_task, cluster_function, NULL);

// init the cluster
pi_cluster_conf_init(&conf);
pi_open_from_conf(&cluster_dev, &conf);

// open the cluster
if (pi_cluster_open(&cluster_dev)) return -1;

// offload an entry point to the cluster
pi_cluster_send_task_to_cl(&cluster_dev, &cluster_task);

// releasing the cluster
pi_cluster_close(&cluster_dev);
```


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – PMSIS – Using the Cluster as a Device

- You will find this content in *main.c*

```
struct pi_device cluster_dev = {0};
struct pi_cluster_conf conf;
struct pi_cluster_task cluster_task = {0};

// task allocation
pi_cluster_task(&cluster_task, cluster_function, NULL);

// init the cluster
pi_cluster_conf_init(&conf);
pi_open_from_conf(&cluster_dev, &conf);

// open the cluster
if (pi_cluster_open(&cluster_dev)) return -1;

// offload an entry point to the cluster
pi_cluster_send_task_to_cl(&cluster_dev, &cluster_task);

// releasing the cluster
pi_cluster_close(&cluster_dev);
```

Prepare PMSIS data structures

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – PMSIS – Using the Cluster as a Device

- You will find this content in *main.c*

```
struct pi_device cluster_dev = {0};
struct pi_cluster_conf conf;
struct pi_cluster_task cluster_

// task allocation
pi_cluster_task(&cluster_task, cluster_function, NULL);

// init the cluster
pi_cluster_conf_init(&conf);
pi_open_from_conf(&cluster_dev, &conf);

// open the cluster
if (pi_cluster_open(&cluster_dev)) return -1;

// offload an entry point to the cluster
pi_cluster_send_task_to_cl(&cluster_dev, &cluster_task);

// releasing the cluster
pi_cluster_close(&cluster_dev);
```

Pointer to a pre-defined function

Create task for cluster

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – PMSIS – Using the Cluster as a Device

- You will find this content in *main.c*

```
struct pi_device cluster_dev = {0};
struct pi_cluster_conf conf;
struct pi_cluster_task cluster_task = {0};

// task allocation
pi_cluster_task(&cluster_task, cluster_function, NULL);

// init the cluster
pi_cluster_conf_init(&conf);
pi_open_from_conf(&cluster_dev, &conf);

// open the cluster
if (pi_cluster_open(&cluster_dev)) return -1;

// offload an entry point to the cluster
pi_cluster_send_task_to_cl(&cluster_dev, &cluster_task);

// releasing the cluster
pi_cluster_close(&cluster_dev);
```

Initialize and open the cluster device

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – PMSIS – Using the Cluster as a Device

- You will find this content in *main.c*

```
struct pi_device cluster_dev = {0};
struct pi_cluster_conf conf;
struct pi_cluster_task cluster_task = {0};

// task allocation
pi_cluster_task(&cluster_task, cluster_function, NULL);

// init the cluster
pi_cluster_conf_init(&conf);
pi_open_from_conf(&cluster_dev, &conf);

// open the cluster
if (pi_cluster_open(&cluster_dev)) return -1;

// offload an entry point to the cluster
pi_cluster_send_task_to_cl(&cluster_dev, &cluster_task);

// releasing the cluster
pi_cluster_close(&cluster_dev);
```

Send and execute the function code on cluster

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – PMSIS – Using the Cluster as a Device

- You will find this content in ***main.c***

```
struct pi_device cluster_dev = {0};
struct pi_cluster_conf conf;
struct pi_cluster_task cluster_task = {0};

// task allocation
pi_cluster_task(&cluster_task, cluster_function, NULL);

// init the cluster
pi_cluster_conf_init(&conf);
pi_open_from_conf(&cluster_dev, &conf);

// open the cluster
if (pi_cluster_open(&cluster_dev)) return -1;

// offload an entry point to the cluster
pi_cluster_send_task_to_cl(&cluster_dev, &cluster_task);

// releasing the cluster
pi_cluster_close(&cluster_dev);
```

Release the cluster

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – Executing on the cluster

- **Fork/join** commands & **SPMD** execution model
- **SPMD** executes **multiple instances** of the **same program independently**, where **each program** works on a **different portion of the data**

```
static void cluster_entry(void *arg)
{
    printf("Hello from cluster\n");
    // ...

    pi_cl_team_fork(NUM_CORES, cores_function, (void *) &args);
    // ...
}
```

Executed on Core 0

Forks the execution of the same function on NUM_CORES cores

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Executing on the cluster: Fork/join + SPMD

```
static void cluster_entry(void *arg)
{
  printf("Hello from cluster\n");
  // ...

  pi_cl_team_fork(NUM_CORES, cluster_fn, (void *) &args);

  // ...
}
```

Data pointer

Function pointer

Executed on core 0

Fork the execution of the same function on NUM_CORES cores

Executed on core 0

SPMD executes multiple instances of the same program independently, where each program works on a different portion of the data

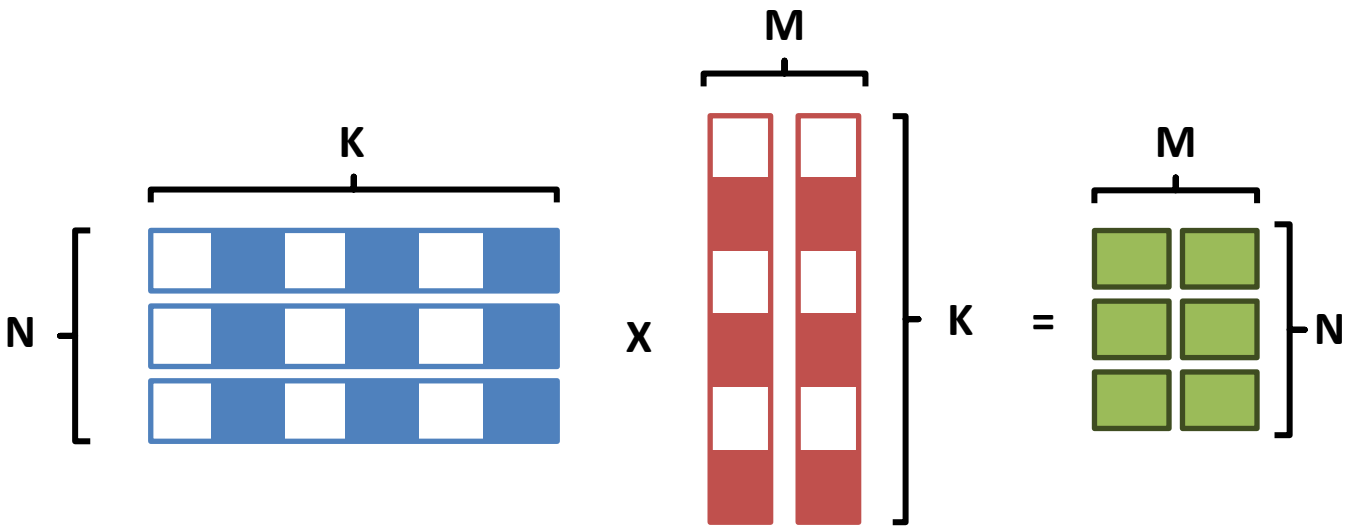
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



04. Tasks

04. Assignments – Task 01: Matrix-Multiplication

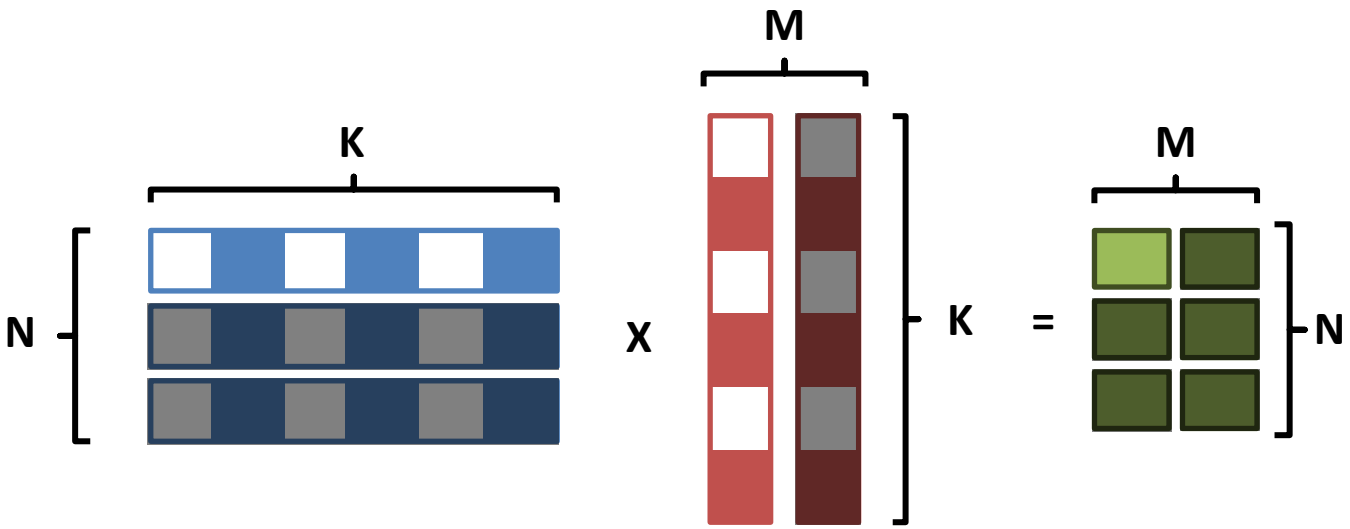
MACs = N*K*M



A X B = C

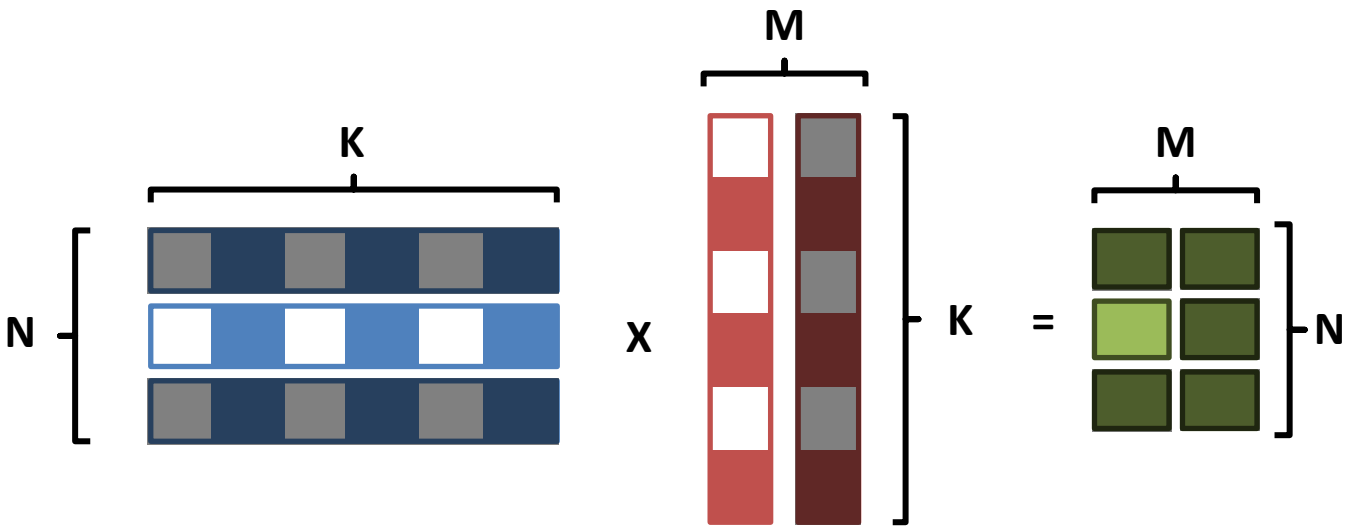
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 01: Matrix-Multiplication



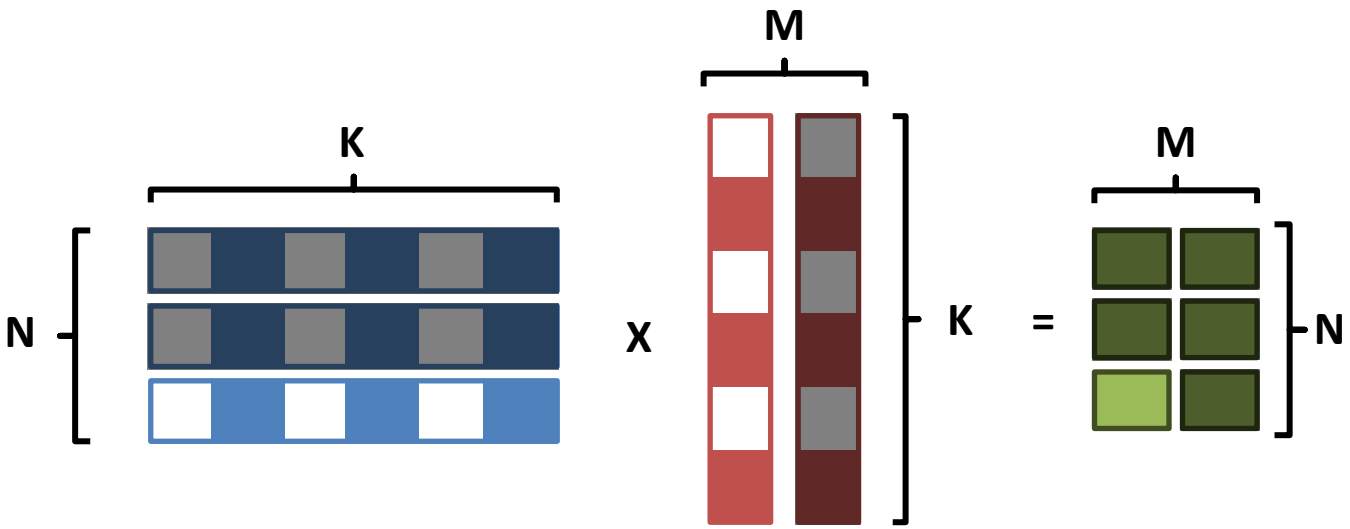
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 01: Matrix-Multiplication



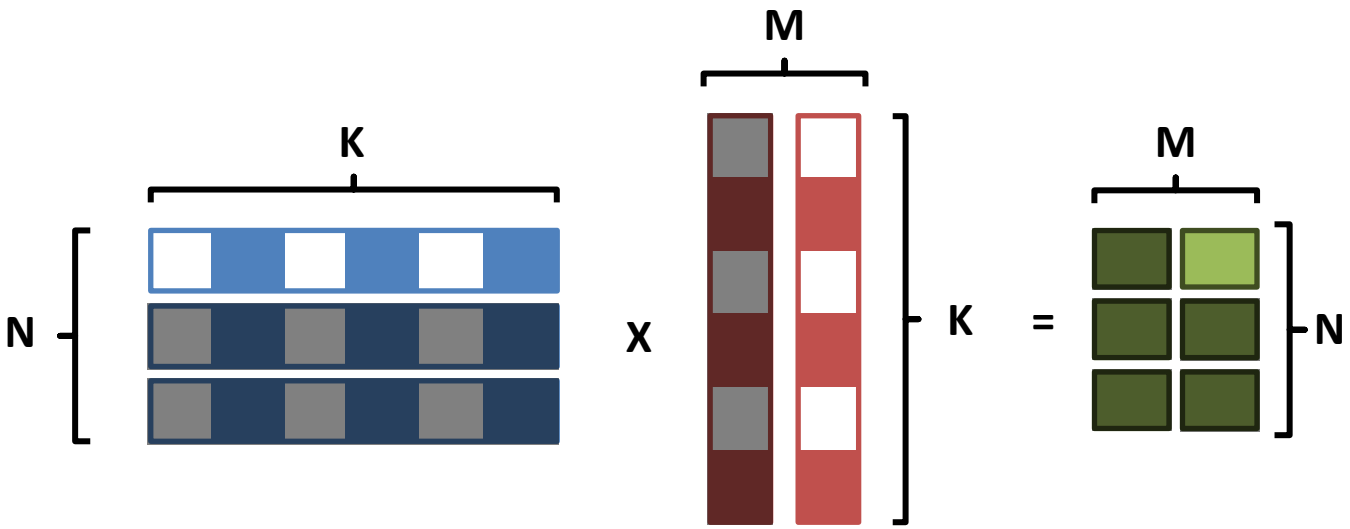
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 01: Matrix-Multiplication



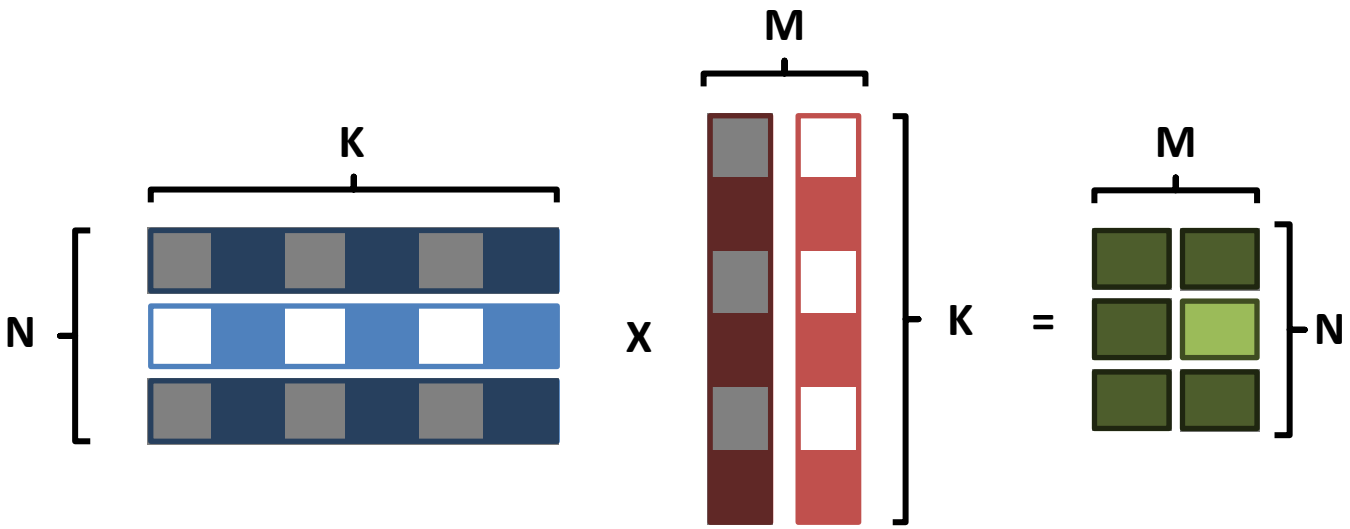
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 01: Matrix-Multiplication



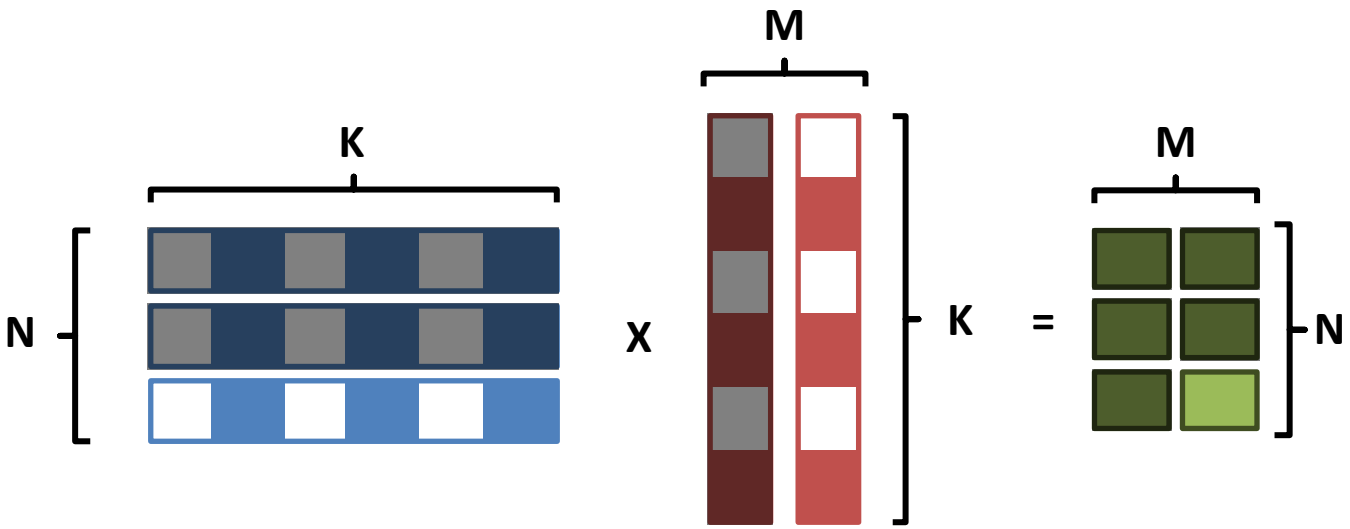
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 01: Matrix-Multiplication



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 01: Matrix-Multiplication



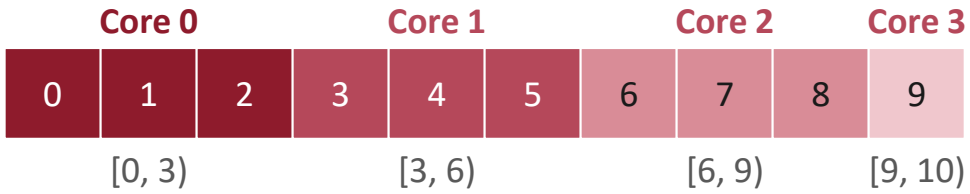
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Tasks 1.1 & 1.2

- Analyze task splitting, speedup and Amdahl's law relative to a varying number of cores and input sizes
- Speedup overview
  - **S(N) = T(1) / T(N)**: execution time on 1 core divided by execution time on N cores
  - Ideal (linear) speedup is S(N) = N; overheads such as synchronization keep real values lower
- Amdahl's law overview
  - **S(N) = 1 / ((1 – p) + p / N)**, where p is the fraction of the program that runs in parallel
  - The serial part caps the gain: as N grows, S(N) approaches 1 / (1 − p)
- How we make sure every core works on a different part of data? Inside the gemm function:

```
core_id = pi_core_id();
i_chunk = (NN + NUM_CORES-1) / NUM_CORES;
i_start = core_id * i_chunk;
i_end = i_start + i_chunk < NN ? i_start + i_chunk : NN;
```

Example: NN = 10, NUM_CORES = 4 → i_chunk = 3



i_end is clamped to NN, so the last core can get fewer elements (core 3 gets 1).

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 1.3 – Load Stalls & Loop Unrolling of the MatMul

- **Load stall**: when an instruction uses a value right after loading it, the core waits for the data and loses cycles
- **Loop unrolling**: run several iterations per loop pass, so independent loads are scheduled before the MACs that need them
  - Fewer branches and index updates per MAC; when K is not a multiple of the unroll factor, a short loop handles the leftover iterations

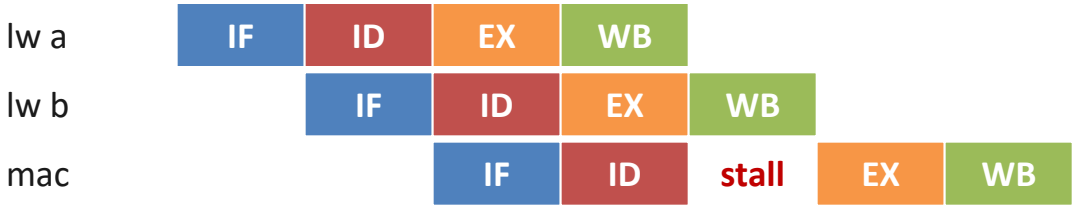
Baseline

```
for (k = 0; k < K; k++)
    acc += A[i*K + k] * B[k*M + j];
```

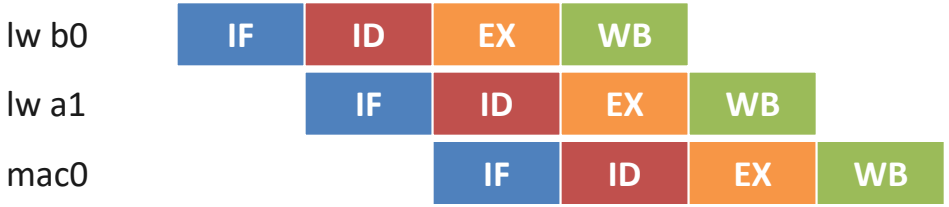
Unrolled ×2

```
for (k = 0; k < K; k += 2) {
    acc += A[i*K + k]     * B[k*M + j];     // mac0
    acc += A[i*K + k + 1] * B[(k+1)*M + j]; // mac1
}
```

Without unrolling: mac waits for lw b **Load stall!**



Unrolled ×2: an independent load fills the gap



Simplified 4-stage pipeline: loaded data is ready after the load's EX

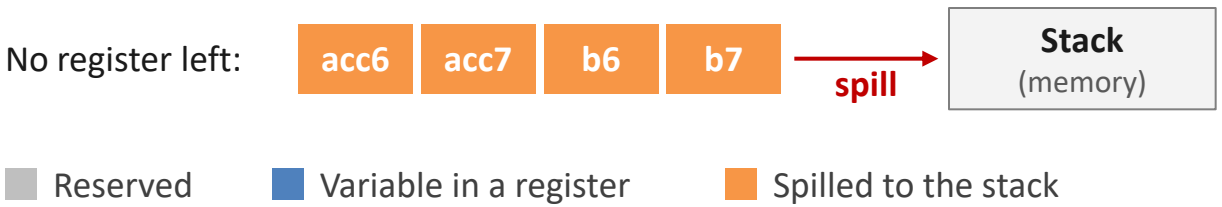
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 1.3 – Unrolling limitations

- **The register file is not infinite**: the core has 32 general-purpose registers, and a few are reserved (zero, ra, sp, gp, tp)
- A wide unrolled loop body needs more live variables (accumulators, loaded values, indices) than there are free registers
- The compiler then **spills** variables to the stack: an extra store and load on every iteration, so **performance drops**

Register file with a ×8 unrolled loop body

zero	ra	sp	gp	tp	x5	x6	x7
x8	x9	x10	x11	x12	x13	x14	x15
x16	x17	x18	x19	x20	x21	x22	x23
x24	x25	x26	x27	x28	x29	x30	x31



Reserved Variable in a register Spilled to the stack

What spilling looks like in the disassembly

```
sw   s0, 28(sp)    # spill to stack
...
lw   s0, 28(sp)    # reload
mul  t0, s0, a1    # use: may stall
```

Look for sp-relative `sw` / `lw` inside the loop: past some unroll factor, cycles go **up** instead of down.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 2 – FullyConnected Parallelization & Leftovers Alignment Issues

Packed SIMD: vectors live in the integer register file

The instruction encodes how to interpret the register content

rs1	0x03	0x02	0x01	0x00
rs2	0x0D	0x0C	0x0B	0x0A

add rD, rs1, rs2	`rD = 0x03020100 + 0x0D0C0B0A`	1× 32-bit ADD
pv.add.h rD, rs1, rs2	`rD[0] = 0x0100 + 0x0B0A` `rD[1] = 0x0302 + 0x0D0C`	**2× 16-bit ADD in 1 cycle**

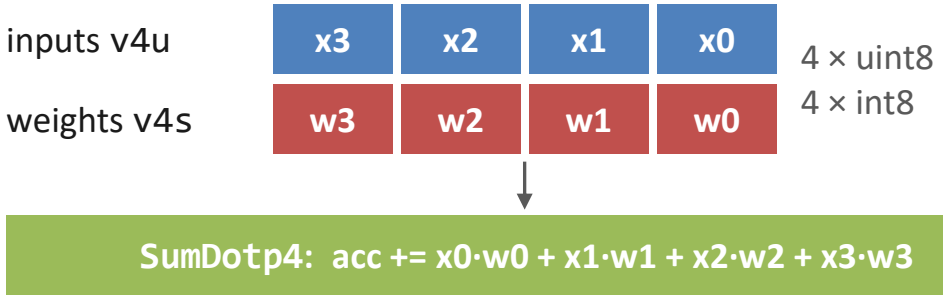
pv.add.b rD, rs1, rs2	`rD[0] = 0x00 + 0x0A`
	`rD[1] = 0x01 + 0x0B`
	`rD[2] = 0x02 + 0x0C`
	`rD[3] = 0x03 + 0x0D`
		4× 8-bit ADD in 1 cycle

pv.* = vector instructions of the Xpulp extension

Note: the MAC count depends on the layer size, not on SIMD vs. non-SIMD

MACs = input size × output neurons

SIMD dot product: 4 MACs in one instruction



Scalar: 4 MAC instructions → SIMD: **1 instruction, 1 cycle**

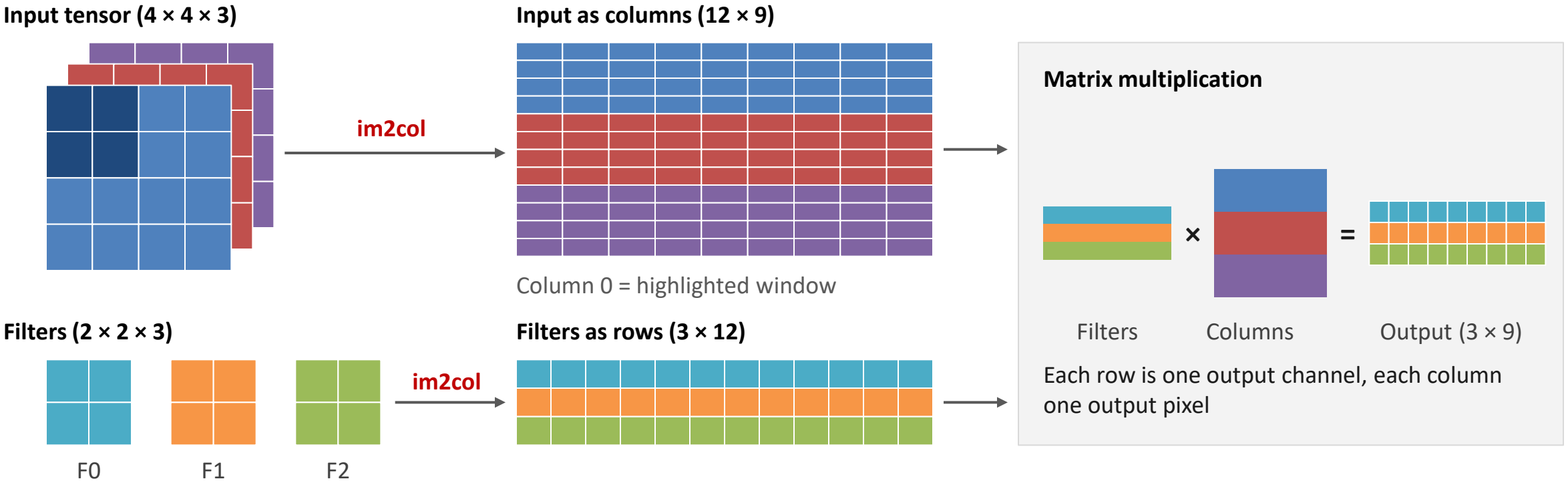
Leftovers: the length is not always a multiple of 4



Example with 10 elements: 2 SIMD steps, then a scalar loop for the last 10 % 4 = 2

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 3 – Convolution Parallelization – Naïve vs im2col + MatMul



im2col turns the convolution into one large MatMul, so the parallel MatMul kernel can be reused

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 3 – Convolution Parallelization – Conv2D on PULP

Parallel loop

```
for i in range(0, H_out):
  for j in range(0, W_out):
    for m in range(0, K_out):
      for ui in range(0, F):
        for uj in range(0, F):
          for n in range(0, K_in):
            im2col[ui*F*K_in + uj*K_in + n] =
                x[i+ui, j+uj, n]
      psum = 0
      for idx in range(0, F*F*K_in):
        psum += w[m, idx] * im2col[idx]
      y[i, j, m] = act(psum)
```

im2col

GEMM

Implementation details

- Standard **output stationary** loop nest
- **HWC** layout for activations, **CoHWCi** for weights
- 2 ***im2col* buffers** per core
- **Unrolled** MatMul

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Călin Diaconu – calin.diaconu2@unibo.it

Ahmad Mirsalari – seyedahmad.mirsalar2@unibo.it

Ihor Sachok – ihor.sachok@unibo.it